USER MANUAL

SOFA

Secure Object Fieldbus Access

Open-source FBsec (FieldBus Security)
for CANopen FD

EmSA-UM-105-COP
Version 1.2, 2026-07-22

Embedded Systems Academy GmbH
Bahnhofstrasse 17
30890 Barsinghausen
Germany

Embedded Systems Academy, Inc.
84 W. Santa Clara St., Suite 700
San Jose, CA 95113
United States

www.em-sa.com

1
Table of Contents

2

Scope and Audience

SOFA (Secure Object Fieldbus Access) demonstrates application-layer secure read and write of CANopen FD object dictionary entries on top of unmodified USDO (Universal Service Data Object) transfers. The package ships source code for a CANopen FD bus simulator, a server and a client, all running on a Windows PC and driven from the command line. An optional bridge to a PEAK PCAN-Basic interface lets the simulated bus carry traffic onto and off of a real CAN FD network for live tracing or interaction.

SOFA is a CiA 720 device. It serves the CiA 720 security object dictionary in the reserved range C000h to C04Fh in two blocks. The AEAD block is the symmetric core: the base descriptors, the AEAD key identifiers and the authenticated identification. The RPK block is the raw-public-key (Ed25519) layer: device identity, ownership control, the signed authenticated identification and signed generic access. The RPK block is a default-on build option, so an out-of-the-box build serves both blocks; setting `FBSEC_FEATURE_ASYM` to 0 produces a minimal AEAD-only device. The X.509 objects are not built yet. Chapter 4 is the normative reference for what is present.

2.1 What This Manual Covers

This manual is the implementation reference for the CANopen FD variant. It states what the code does, exactly, in the places where the whitepapers deliberately stop short:

- Chapters 4 and 5 are normative. They fix every object dictionary entry the reference server implements and every byte the demonstrator puts on the wire.
- Chapters 6 through 8 cover building, running and driving the three programs.
- Chapters 9 through 11 are the porting guide for taking SOFA onto a real device.
- Chapter 12 states what the cryptography defends against and what it does not.

Appendix D maps the wider document set and says which document owns which question.

For the wider context that SOFA sits in, EmSA maintains a CAN security reference at cansecurity.net. The overview of this protocol is at cansecurity.net/solutions/secure-object-fieldbus-access, and the glossary at /resources/terms-and-definitions defines the vocabulary this manual assumes.

2.2 Prerequisites

Two whitepapers set the context, and reading them first will save time:

- EmSA-WP-105, “Secure Object Fieldbus Access”, gives the conceptual model and the protocol shape that SOFA implements.
- EmSA-WP-104, “Key Provisioning for Minimal Fieldbus Systems”, gives the key lifecycle that the simulator skips but that a real deployment must implement.

The reader of the porting chapters is assumed to have AEAD basics, in particular why nonce reuse is fatal under GCM, and practical CANopen FD experience with a USDO-capable stack.

To build and run the demonstrator you need:

- Windows 10 or 11.
- CMake 3.20 or later.

- MSVC 14.50 (Visual Studio 2026). The tree builds clean under `/W4 /WX /permissive-`. The bundled executables statically link the MSVC C runtime, so they run on any Windows 10 or later machine without the Visual C++ Redistributable.

No separate cryptographic library install is required. A seven-file mbedtls subset covering AES, GCM and SHA-256 is vendored under `third_party/mbedtls/`, and the default asymmetric build additionally vendors Monocypher for Ed25519. A minimal AEAD-only build (`FBSEC_FEATURE_ASYM=0`) drops the Monocypher dependency.

Live CAN FD bridging is optional and needs:

- The PEAK PCAN-Basic Windows driver, PEAK-Drivers 5 or newer.
- A CAN FD-capable PEAK interface such as PCAN-USB FD, PCAN-USB Pro FD or PCAN-PCI Express FD. Classical-CAN-only adapters are not sufficient, because SOFA USDO frames carry up to 64 payload bytes.

`PCANBasic.dll` is resolved at run time through `LoadLibrary`. Machines without it still build and run the simulator; only the bridge feature is skipped.

2.3 What This Demonstrator Is Not

Key handling in the simulation follows the WP-104 vocabulary but skips the lifecycle. WP-104 specifies that each layer, meaning Provisioning, Integrator and Operator, holds a base key, and that a per-session key is derived per access through HKDF. This implementation skips the derivation: what `--key-file` loads and what travels on the wire are the already-derived session keys for each role.

> **CAUTION:** This package is a simulator and a demonstrator, not a secure product. It implements no provisioning state machine, no non-volatile key storage, no authenticated key install or overwrite, and no manufacturer reset. A real embedded target must implement that lifecycle in its port layer. Do not ship the simulator key handling as production security.

2.4 Portability

The variant-agnostic layers, namely `shared/` for the cryptography and the state machine, plus `client_common/`, `server_common/` and `bus_common/`, are written so that a future variant such as CANopen CC or EtherCAT can drop into `variants/<name>/` without touching the common code. The four secure verbs and the object-dictionary registry are unchanged across variants; only the address mapping and the transport codec are variant-specific.

3
Concepts and Terminology

This chapter defines the terms the rest of the manual uses. It is deliberately short. EmSA-WP-105 develops the model; the purpose here is to fix vocabulary so that chapters 4 and 5 can be read without ambiguity.

What SOFA sets out to achieve is narrow, and worth stating before the vocabulary:

- **Secure read and write access to selected object dictionary entries**, based on symmetric cryptography. Each access is authenticated and replay-resistant, and confidential when encryption is enabled. Which entries are secure is the integrator's choice; the rest of the dictionary is untouched.
- **An optimized path for repetitive access**, so that a value polled or pushed continuously does not pay the full challenge-response cost on every frame.
- **Optionally, manufacturer-to-integrator handover**, based on asymmetric cryptography. Its purpose is to establish who owns a device and to install the symmetric keys that the first two objectives then rely on.

Everything else follows from those three. The symmetric tunnel is the product; the asymmetric layer exists to get its keys in place and to sign high-value accesses. It is built in by default, and a minimal AEAD-only build omits it.

The security objects a device advertises live in the CiA 720 reserved range C000h to C04Fh. This release serves the AEAD subset of that range; chapter 4 is the normative list.

3.1 The Secure Tunnel

A tunnel is a server-side data object. Clients send requests to it and the server emits responses from it. The secure-tunnel layer makes those exchanges confidential when encryption is enabled, authenticated, and replay-resistant, without changing the host fieldbus framing or addressing.

The model is unilateral, which is to say client-driven. The client picks the data object, the verb, and the encryption mode; the server validates, applies role policy, and answers. There is no separate session-establishment handshake. Every request carries the keying inputs it needs, and the authentication tag binds the device id, the data id and the direction.

The party that verifies the tag is the party that learns something. On a read that is the client, which thereby authenticates the server; on a write it is the server, which thereby authenticates the client. Both peers still contribute fresh randomness to every single-shot transfer.

3.2 Addressing

Two identifiers appear in every exchange and in the associated data of every tag.

Term	Meaning
`device_id`	Which device. On CANopen FD this is the node id, 1 to 127, promoted to 16 bits. The server's `device_id` is its own node id; the client names the target server's node id.

Term	Meaning
`data_id`	Which object. On CANopen FD this is the index and subindex pair packed into 32 bits. Chapter 4 gives the encoding.

The CAN identifier separately encodes the sender's node id in its low seven bits. The CAN-encoded sender and the receiver node id in the frame together drive routing on a real bus; binding `device_id` into the associated data is what makes that routing tamper-evident.

3.3 The Four Verbs

The demonstrator implements four secure verbs. The distinction between them is a client-side choice, not a property of an entry: every read-capable entry supports both single and cyclic reads, and every write-capable entry supports both single and cyclic writes.

Verb	Name	Shape
`srd`	Secure read	Two passes. The client challenges, the server answers with data and a tag.
`swr`	Secure write	Two passes. The server challenges, the client sends data and a tag.
`srdpoll`	Cyclic secure read	One cyclic-capable `srd` to arm a session, then repeated single-frame poll reads.
`swrpoll`	Cyclic secure write	One cyclic-capable `swr` to arm a session, then repeated single-frame poll writes.

3.4 Single and Cyclic Modes

A cyclic access is not a different kind of entry. It begins with exactly the same two-pass challenge and response that a single access uses, with one extra bit in the request asking the server to keep the session alive for follow-up polls against that address.

Cyclic mode amortizes the per-frame challenge cost. Follow-up polls are one request and one response carrying only a counter byte plus ciphertext and tag. It suits fast liveness or freshness verification of a read-only entry, and streaming a slowly-changing setpoint to a write-only entry. It does not suit one-shot configuration changes, where a single write gives a cleaner audit trail, and it does not suit publish-subscribe, because SOFA is strictly client-server with exactly one client and one server per session.

A cyclic session ends when it goes idle past the timeout, when the per-key frame budget is reached, or when the client stops. Chapter 4 gives both limits. Recovery is always the same: arm a fresh session with another cyclic-capable single access.

3.5 Encryption Versus Authentication Only

Encryption is a per-access runtime choice, carried in bit 7 of the wire key selector and covered by the tag.

- With encryption on, the entry data travels as ciphertext and the tag authenticates it.

- With encryption off, the entry data travels as plaintext, is included in the associated data, and the tag still authenticates it. This mode exists for deployments that need integrity and freshness but must keep the payload readable to existing tooling.

Downgrading is not possible in flight. The full key selector byte sits in the associated data, so clearing bit 7 on a captured frame fails tag verification.

3.6 Key Roles

Three roles carry the demonstration policy, following the WP-104 layer names.

Role	Key id	Intent
Provisioning Key	1	Factory and integrator install.
Integrator Key	2	Site deployment.
Operator Key	3	Runtime operation.

The cryptographic core does not interpret these numbers. Only the role hook does, and chapter 4 states the policy it enforces. Chapter 10 covers where the keys come from and how they are stored.

> **NOTE:** What the demonstrator loads and installs are session keys, already derived. WP-104's base keys and the HKDF step that turns a base key into a session key are simulated out. The role names identify which WP-104 layer's base key would have produced the session key on a real device, not what is stored here.

4

Implemented Object Dictionary

This chapter is the normative object-dictionary reference for the SOFA CANopen FD demonstrator. It states every index and subindex the reference server implements, the exact values it serves, and the two gates that decide whether an access is permitted. Everything here is fixed by the code rather than by EmSA-WP-105, so this manual is the only place it is written down.

SOFA implements the CiA 720 security object dictionary, which reserves the range C000h to C04Fh for security objects, in two blocks. The AEAD block is functional: the base descriptors C000h and C001h, the AEAD key identifiers C011h and the authenticated identification C018h. The session salt C010h and the key set C01Fh are present but return a not-implemented abort until a later pass adds the salt-armed session layer and over-the-bus key install. The raw-public-key (RPK) block is now functional in the default build: ownership control C020h, the public keys C021h and their types C022h, the signed authenticated identification C028h, the signed provisioning-key install C02Fh, the signed generic access C042h and the signed function command C049h. Setting `FBSEC_FEATURE_ASYM` to 0 compiles the RPK block out for a minimal AEAD-only device. The X.509 objects are not implemented yet. The migration decision is recorded in `doc/decisions/adr-001-cia720-aead-migration-pass1.md`, and the target dictionary in `doc/decisions/cia720-security-od.md`.

4.1 Address Mapping

SOFA carries a 32-bit `data_id` in its associated data. On a CANopen FD deployment the encoding is:

```
data_id = ((index & 0xFFFF) << 16) | ((sub & 0xFF) << 8)
```

The low byte stays `0x00`. It is reserved for per-bus protocol fields that some industrial buses carry alongside the address. A decoder must reject any frame whose `data_id` low byte is non-zero, and must reject an index below `0x1000`, which CiA-1301 reserves for protocol use. The reference codec enforces both rules in `fbsec_co_fd_data_id_is_canopen_form()`; a violation is reported on the simulated bus as a `SIM_FRAME_ERROR` with code `0x06`.

Three helpers in `variants/canopen_fd/common/fbsec_co_fd_addr.h` implement the mapping:

- `fbsec_co_fd_data_id_from_index_sub()` builds a `data_id`.
- `fbsec_co_fd_index_sub_from_data_id()` extracts the index and subindex.
- `fbsec_co_fd_data_id_is_canopen_form()` validates an inbound wire `data_id`.

On the command line the client requires the full 32-bit value, including the reserved low byte: `srd 5 0xC0180000`. Passing the 24-bit short form is rejected with `invalid CANopen-form data_id`. Traces and the interactive menu, in contrast, display addresses in 24-bit form with the reserved byte dropped, so `0xC01800` on screen is `data_id 0xC0180000` on the command line and on the wire.

4.2 Dispatch Order

The server answers a request by walking four dispatch tiers in order, and the first tier that owns the `data_id` handles it:

1. The base descriptors C000h and C001h, served read-only and unauthenticated by the descriptor module.

2. The constant, unsecured entries loaded from the object-dictionary file, object 1018h above all, served read-only.
3. The CiA 720 AEAD-block objects C010h, C011h and C01Fh, served by a dedicated security handler.
4. The secure-object registry, which holds C018h and the application demonstration entries.

An asymmetric build inserts the RPK objects (C020h, C021h, C022h, C028h, C02Fh, C042h and C049h) between the security handler and the registry, where a dedicated RPK handler serves them. A `data_id` that no tier owns aborts with `FBSEC_ABORT_NO_OBJECT` (`0x33`).

4.3 Secure Data Entries

The reference server registers up to six secure entries at start-up, in the order shown. The last two exist only in an asymmetric build. All are registered with `key_id = FBSEC_SOD_KEY_NONE`, so the AEAD step accepts any provisioned key and the read or write decision is left entirely to the role policy described under Access Control below.

data_id	Index, sub	Access	Size	Purpose
`0xC0180000`	0xC018, 0x00	SECURE_RO	16 bytes	Authenticated identification: the object 1018h identity quad
`0x20160000`	0x2016, 0x00	SECURE_WO	16 bytes	Free-form write target, last write kept
`0x20200000`	0x2020, 0x00	SECURE_RO	4 bytes	Auto-incrementing 32-bit counter
`0x20100000`	0x2010, 0x00	SECURE_WO	4 bytes	Write target, mirrors into 0x2020
`0x20210000`	0x2021, 0x00	SECURE_RO	4 bytes	RPK read twin of 0x2020, reached through C042h signed read
`0x20170000`	0x2017, 0x00	SECURE_WO	16 bytes	RPK write twin of 0x2016, reached through C042h signed write

C018h is the CiA 720 authenticated-identification object; the other entries sit in the manufacturer-specific application range and demonstrate the secure verbs. All are declared as `FBSEC_SOD_TYPE_BIN`, which the trace renders as hex, and all travel with the USDO data-type code `DOMAIN` (`0x0F`). The two RPK twins 0x2021 and 0x2017 exist so that the same read and write operation can be shown under the RPK mechanism beside the AEAD one: 0x2021 mirrors the 0x2020 counter and 0x2017 mirrors the 0x2016 write target, each reachable only through the signed generic access object C042h described later in this chapter.

C018h is registered only when its backing identity is loaded. Its 16 bytes are the object 1018h quad, vendor id, product code, revision and serial, copied at start-up from the constant object-dictionary table by `fbsec_server_hooks_load_identity()`. Without an `--od-file` that supplies 1018h, the entry is absent and a read of C018h aborts with `FBSEC_ABORT_NO_OBJECT` (`0x33`). A secure read returns the 16 identity bytes followed by the AEAD tag under the session key. The signed (RPK) flavor of this object, C028h, is described under the RPK objects later in this chapter.

The values the other entries hold at start-up, and how they change, are fixed by the reference port hooks in `server_common/server_common_hooks.c`:

- `0x2016, 0x00` starts as sixteen zero bytes and holds whatever was written last.
- `0x2020, 0x00` and `0x2010, 0x00` share one 4-byte buffer whose initial value is `00 11 22 33`. A read of `0x2020` returns the current value and then increments it as a big-endian 32-bit integer, so

successive reads tick. A write to `0x2010` overwrites the same buffer, which is why the next read of `0x2020` returns the value just written.

- `0x2021, 0x00` mirrors the 0x2020 counter under the RPK mechanism: a signed read returns the current value and increments it the same way.
- `0x2017, 0x00` mirrors the 0x2016 write target under the RPK mechanism: a signed write keeps the last sixteen bytes.

Write length is checked exactly: sixteen bytes for `0x2016` and `0x2017`, four bytes for `0x2010`. A mismatch aborts with `FBSEC_ABORT_LEN_TOO_HIGH` (`0x37`) or `FBSEC_ABORT_LEN_TOO_LOW` (`0x38`). A read or write against a `data_id` the hooks do not recognize aborts with `FBSEC_ABORT_NO_OBJECT` (`0x33`).

> **NOTE:** These are the only entries the secure-object registry holds, and the demonstrator implements no plain read or write access to application data: the CANopen FD client rejects the `rd` and `wr` verbs, so only the four secure verbs reach an application entry. The base descriptors, the constant 1018h identity and the AEAD key identifiers described next are the deliberate exception. They are read cold, before any key exists, because a client has to be able to learn what a device is before it can talk to it securely.

4.4 Base Descriptors C000h and C001h

Two read-only records let a client learn what a device supports before any key or identity exists. They are the CiA 720 base parameters: C000h is the security profile and capabilities, C001h the security status. Both are unauthenticated. The server answers them ahead of every other dispatch tier, with no key, no tag and no challenge, so any USDO-capable tool can read them cold. C000h is computed from the build so it always mirrors the real configuration; C001h reflects live commissioning and key-slot state. Neither is stored in the object-dictionary file.

C000h Security Profile and Capabilities

C000h is an array of nine subindices, following the CiA 720-2 layout. A tool reads the algorithms from the individual subindices and must never infer them from the profile number.

Sub	Type	Field	Value in the default build
0x00	U8	Highest subindex supported	`0x08`
0x01	U32 LE	Security type word	`0x00000000`
0x02	U32 LE	AEAD algorithms and tag lengths	`0x00001001` (AES-128-GCM, 16-byte tag)
0x03	U16 LE	Key derivation functions	`0x0001` (HKDF-SHA-256)
0x04	U16 LE	Signature algorithms	`0x0001` (Ed25519)
0x05	U8	Symmetric key levels supported	`0x07` (Provisioning, Integrator, Operator)
0x06	U8	Asymmetric key presence	`0x01` (IDevID; LDevID appears after export)
0x07	U8	Claim gates supported	`0x05` (TOFU and ownership voucher)
0x08	U16 LE	Mechanisms implemented	`0x0003` (AEAD and RPK)

The default build compiles in the RPK block, so C000h advertises it. A minimal AEAD-only build (`FBSEC_FEATURE_ASYM` set to 0) reports the same security type word `0x00000000`, no signature algorithm (`0x0000`), no asymmetric key (`0x00`), the claim gates `0x01` and the mechanisms word `0x0001` (AEAD alone).

The security type word at subindex `0x01` mirrors CiA 1301 object 1000h field for field.

Bits	Field
11:0	Security profile number: 000h Custom, 001h Open, 002h Claimed, 003h Authorized, 004h Identified, 005h Confidential
15:12	Capability level, 0 to 3 (C0 to C3)
31:16	Additional information, defined by the reported profile

The default value `0x00000000` reads as profile Custom, capability level C0. The profile is Custom because a deliberately partial device makes no numbered-profile conformance claim yet, and the level is C0 for this release. The supported mechanisms no longer live in the type word; they are reported separately in subindex `0x08`, which reads `0x0003` (AEAD and RPK) in the default build and `0x0001` (AEAD alone) in a minimal build. The X.509 bit is never set in this release.

The remaining subindices carry bitmaps and identifiers, each bit following the CiA 720-1 algorithm registry:

Field	Bit or code values
AEAD algorithms, sub 0x02 low byte	AES-128-GCM `0x01`, AES-256-GCM `0x02`, Ascon-128 `0x04`, ChaCha20-Poly1305 `0x08`. High byte is the tag length in bytes.
Key derivation functions, sub 0x03	HKDF-SHA-256 `0x0001`
Signature algorithms, sub 0x04	Ed25519 `0x0001`
Symmetric key levels, sub 0x05	Provisioning `0x01`, Integrator `0x02`, Operator `0x04`
Asymmetric key presence, sub 0x06	IDevID `0x01`, LDevID `0x02`, X.509 `0x08`
Claim gates, sub 0x07	TOFU, claim on first use `0x01`, device claim token `0x02`, ownership voucher `0x04`
Mechanisms implemented, sub 0x08	AEAD `0x0001`, RPK `0x0002`, X.509 `0x0004`

The type word and the algorithm subindices are separate on purpose. Capability is declared in the type word; subindex `0x06` reports which identity artifacts are loaded right now, and subindex `0x02` reports the AEAD primitive and tag length the build actually runs.

C001h Security Status

C001h is the security status object, following the CiA 720-2 layout. It reports the active configuration, holds no key material or nonces, and stays small since it is readable cold.

Sub	Type	Field	Value in the demo build
0x00	U8	Highest subindex supported	`0x05`
0x01	U8	Commissioning state: 0 uncommissioned, 1 commissioned/ owned	`0x01`
0x02	U8	Active claim gate (one of the supported gates)	`0x04` (ownership voucher)
0x03	U8	Base keys installed: b0 Provisioning, b1 Integrator, b2 Operator	`0x07` (all three)
0x04	U32 LE	Active AEAD algorithm and tag length	`0x00001001`
0x05	U8	Device identities present: IDevID `0x01`, LDevID `0x02`	`0x01`

The demo ships with the three demonstration keys loaded and runs as if a voucher handover had happened in the past, so it reports commissioning state 1, the active claim gate, and all three key bits set. Removing a key from the store clears its bit in subindex `0x03`.

Reading and Refusing the Descriptors

Both records are read-only and bounded. Addressing is checked before access: a subindex above the highest supported, or a `data_id` whose reserved low byte is non-zero, is refused with `FBSEC_ABORT_NO_SUBINDEX` (`0x34`). Only a valid subindex that then carries a body, which is to say a write attempt, is refused with `FBSEC_ABORT_READ_ONLY` (`0x32`).

Reading them needs no key and no SOFA client. A plain USDO upload request to the index and subindex, carrying no data, returns the value. The client's `caps` subcommand does the same and decodes C000h; it is built into every configuration. The interactive menu's scan action reads both records plus C011h and object 1018h and prints every subindex with its meaning; chapter 7 shows it.

Because the mechanism byte and the key selector are bound into the associated data on every secure verb, a spoofed descriptor cannot force a peer into a weaker mode. A client that reads a descriptor is expected to fail closed and refuse a mode the device does not advertise.

4.5 AEAD Key Identifiers C011h

C011h is a read-only, unsecured array of the non-secret key identifiers, for discovery. Subindex `0x00` is the slot count, four here, matching `FBSEC_SOD_KEY_SLOTS`. Subindices `0x01` and above carry one U32 key id or version per slot, served little-endian. A slot that holds no key reads `0x00000000`. The key id is independent of the slot number, so a tool can see which key version a device runs without reading any secret.

Sub	Type	Field	Value in the demo build
0x00	U8	Highest subindex supported (slot count)	`0x04`
0x01	U32 LE	Provisioning key id	`0x00001001`

Sub	Type	Field	Value in the demo build
0x02	U32 LE	Integrator key id	`0x00001002`
0x03	U32 LE	Operator key id	`0x00001003`
0x04	U32 LE	Application key id	`0x00000000` (empty)

The demonstration sets the three ids to `0x1001`, `0x1002` and `0x1003` deliberately, so that C011h visibly reports an id different from the slot and role number. A write to C011h is refused with `FBSEC_ABORT_READ_ONLY` (`0x32`), and a subindex above the slot count with `FBSEC_ABORT_NO_SUBINDEX` (`0x34`).

4.6 Surface-Only Objects C010h and C01Fh

Two AEAD-block objects are present in the dictionary so a conformance tool sees the correct CiA 720 surface, but their function is deferred to a later pass. Both live in the security handler.

C010h is the session pre-requisites object. Subindex `0x00` reads highest-subindex `0x01`, and subindex `0x01` reads a 16-byte all-zero session salt. A read is honest, but arming a session by writing a salt is not implemented yet: a write to subindex `0x01` returns the new abort `FBSEC_ABORT_NOT_IMPLEMENTED` (`0xC9`).

C01Fh is the single write-only key-set slot for over-the-bus key install and rotation. A read is refused with `FBSEC_ABORT_WRITE_ONLY` (`0x31`), and a write, which would install a key, returns `FBSEC_ABORT_NOT_IMPLEMENTED` (`0xC9`). A subindex other than `0x00` returns `FBSEC_ABORT_NO_SUBINDEX` (`0x34`).

> **NOTE:** `0xC9` NOT_IMPLEMENTED means the object exists in the dictionary but its function is not built yet. It is distinct from `0xC8` NOT_BUILT, which means a whole feature was compiled out. A tool that reaches C010h or C01Fh sees the right layout and an honest refusal on use, not a missing object.

4.7 Constant Object Dictionary and Object 1018h

Object 1018h, the CANopen identity, and any other constant, unsecured, readable entries are served from a small table loaded at start-up with the server flag `--od-file`. Each line of the file is one entry in hex tokens, and `#` starts a comment:

```
<index> <sub> <len> <data-byte> ...
```

The table holds up to `FBSEC_CONST_OD_MAX_ENTRIES` (32) entries of up to `FBSEC_CONST_OD_MAX_DATA` (8) bytes each. A read of a listed entry returns its bytes exactly as authored, big-endian for the 1018h U32 values; a write to one is refused with `FBSEC_ABORT_READ_ONLY` (`0x32`). C000h and C001h are never placed in this file: they stay build-computed and live-computed so a tool always reads the real configuration.

The shipped `run/sofa-od-demo.txt` defines object 1018h, and `start_fd_server.bat` passes it. The four identity U32 values are:

Sub	Field	Value	Bytes
0x01	Vendor id	`0x000002F1`	`00 00 02 F1`
0x02	Product code	`0x534F4641` ("SOFA")	`53 4F 46 41`

Sub	Field	Value	Bytes
0x03	Revision	`0x00010001`	`00 01 00 01`
0x04	Serial number	`0x00000001`	`00 00 00 01`

C018h reads these sixteen bytes back under an AEAD tag, so on the demo build its plaintext is `00 00 02 F1 53 4F 46 41 00 01 00 01 00 00 00 01`.

> **CAUTION:** The vendor id `0x000002F1` is a demonstration placeholder, not a CiA-assigned vendor id. Replace it with your assigned vendor id for a real device. The constant object-dictionary file is kept in sync with the device by hand for the entries it owns; C000h is exempt because it stays build-computed.

4.8 RPK Objects

The raw-public-key block carries device identity, the manufacturer-to-integrator handover and the signed accesses. Every object in it exists only when `FBSEC_FEATURE_ASYM` is set to 1, which is the default, and the voucher and owner-epoch operations additionally require `FBSEC_HANDOVER_AUTHORIZED`, also on by default with the asymmetric layer. Chapter 11 describes the handover flow; this section fixes the addresses and the exact payload sizes.

Protection in this block is by replacement. A signed access carries no AEAD tag; an Ed25519 signature stands in its place. An object is protected either by AEAD or by a signature, never both. Freshness comes from a two-pass challenge: the server contributes a fresh nonce that the signed transcript covers, and that challenge is also the denial-of-service gate, since a node verifies a signature only against a challenge it issued itself. A signed read has the server sign the value together with the challenge under the device IDevID, and the client verifies against the device public key. A signed write or command has the client sign under its integrator key, and the device verifies against the installed integrator public key.

Ownership Control C020h

C020h holds the ownership-lifecycle operations. Subindex `0x00` reads highest-subindex `0x03`.

Sub	Access	Mechanism	Request	Reply
0x01	ro	none	empty	4 bytes, the owner epoch U32
0x02	rw	RPK	108 bytes: `serial[8]`, `integrator_pub[32]`, `epoch[4]`, `sig[64]`	empty acknowledgement
0x03	ro	RPK	empty challenge	96 bytes: `ldevid_pub[32]`, `sig[64]`

Subindex `0x01` is the owner epoch, a monotonic `U32` that must strictly increase and that survives a manufacturer reset, so a device cannot be rolled back to a previous owner. Subindex `0x02` is the ownership voucher claim, the authorized handover model: the device verifies the manufacturer signature against its trust anchor, installs the integrator public key into C021h and records the voucher owner epoch. Subindex `0x03` triggers on-device LDevID key generation and exports the new public key signed by the IDevID. The voucher and the owner epoch require `FBSEC_HANDOVER_AUTHORIZED`.

Public Keys C021h and Types C022h

C021h is the read-only store of the verification public keys the device holds. Subindex `0x00` reads highest-subindex `0x02`, subindex `0x01` is the manufacturer trust anchor and subindex `0x02` is the integrator public key. The integrator key reads as thirty-two zero bytes until a voucher claim at C020h:02h installs it. Each key is an Ed25519 public key, thirty-two bytes. The keys are read cold and are never written directly over the bus.

C022h reports the type of each key in C021h so a tool need not guess. Subindex `0x00` reads highest-subindex `0x02`, and one `U32` per key follows: byte 0 is the algorithm id, `0x01` for Ed25519, and byte 1 is the key length in bytes, `0x20` for thirty-two.

Signed Authenticated Identification C028h

C028h is the signed flavor of C018h, a challenge-response proof of genuineness. The client writes a fresh 16-byte challenge `rT`, and the device returns 168 bytes in three parts: the 40-byte identity blob (`serial[8]` then the IDevID public key), a 64-byte manufacturer signature over the blob, and a 64-byte device signature over the blob and the challenge.

Sub	Access	Mechanism	Request	Reply
0x00	ro	RPK	`rT[16]` challenge	168 bytes: `blob[40]`, `mfg_cert[64]`, `sig[64]`

The identity blob is `serial[8]` followed by the IDevID public key, forty bytes in total. An identity read whose challenge is not exactly sixteen bytes aborts with `FBSEC_ABORT_LEN_TOO_LOW` (`0x38`). A signature that fails to verify aborts with `FBSEC_ABORT_SIG_VERIFY` (`0xC1`), and a rejected voucher aborts with `FBSEC_ABORT_VOUCHER` (`0xC2`).

Signed Provisioning-Key Install C02Fh

C02Fh is the signed bootstrap that installs the symmetric Provisioning Key on a device that holds an identity but no shared key yet. The write-only payload is `key[16]` then `sig[64]`, eighty bytes, and the device answers with an empty acknowledgement. It is the RPK-gated counterpart of the symmetric key set C01Fh.

Sub	Access	Mechanism	Request	Reply
0x00	wo	RPK	80 bytes: `key[16]`, `sig[64]`	empty acknowledgement

Signed Generic Access C042h and Command C049h

C042h is the RPK sibling of the AEAD generic access C041h. It reads or writes an application entry under a signature rather than a tag. Both directions are a two-pass challenge-response: the device issues a fresh random, and the signature covers a transcript that binds both peer ids, the object multiplexor, the value length, both challenge randoms and the value. Subindex `0x01` is a signed read: after the challenge, the device returns the target value plus an Ed25519 signature, verified by the client against the device public key. Subindex `0x02` is a signed write: the client sends the value plus a signature over the same transcript shape, and the device verifies it against the installed integrator public key before applying the write. In the demonstration the signed read reaches 0x2021 and the signed write reaches 0x2017.

Sub	Access	Mechanism	Field
0x01	ro	RPK	Signed generic read
0x02	wo	RPK	Signed generic write

C049h is the RPK sibling of the AEAD function command C048h, for high-value commands such as manufacturer reset. Subindex `0x00` is a write-only `U32` command carried with a client Ed25519 signature over a transcript that binds the command and both challenge randoms, verified against the installed integrator public key.

Every RPK payload except the owner-epoch read and the plain command exceeds what a single expedited USDO frame carries, so they travel as CiA-1301 USDO segmented transfers; chapter 5 covers that.

4.9 Registry Limits and Timeouts

The secure-object registry is a fixed-size static structure. Its limits bound what an integrator can register without editing headers.

Constant	Value	Meaning
`FBSEC_SOD_MAX_ENTRIES`	8	Registered secure entries. Registering the same `data_id` twice replaces the entry in place rather than consuming a second slot.
`FBSEC_SOD_KEY_SLOTS`	4	Server-side key slots, so key ids 1 through 4 only. The client-side store holds ids 1 through 15. C011h reports this slot count.
`FBSEC_AEAD_MAX_PROTECTED`	32	Largest plaintext an entry may declare. Registration fails above it.
`FBSEC_SOD_CHALLENGE_TIMEOUT_MS`	5000	How long a Pass-1 challenge stays valid before Pass 2 must arrive.
`FBSEC_SOD_SESSION_IDLE_TIMEOUT_MS`	60000	How long a cyclic session survives without traffic.
`FBSEC_AEAD_KEY_USE_LIMIT`	1 000 000	AEAD invocations allowed per key and session before a re-arm is required.

A stale challenge, an expired session and an exhausted frame budget all surface as `FBSEC_ABORT_NO_SESSION` (`0xC5`), and an exhausted budget as `FBSEC_ABORT_KEY_BUDGET` (`0xC7`). The server keeps one armed slot per entry and direction, not one per client, so a second client arming the same entry in the same direction displaces the first.

> **CAUTION:** The server key store has four slots, yet the wire key selector is four bits wide and the client store holds fifteen keys. A key file row with a key id of 5 or above loads cleanly on the client but makes the server refuse to start, reporting `key-file: keyid 5 rejected`. A client that then uses that key id is refused with `0xC4`. Keep key files inside the range 1 to 4.

4.10 Access Control

Two independent gates guard every secure request that reaches a registered entry, and they fail with different abort codes so the cause is unambiguous.

The first gate is cryptographic. The AEAD verification confirms that the request carries a valid authentication tag computed with one of the loaded session keys. A forged or tampered request fails here with `FBSEC_ABORT_TAG_VERIFY` (`0xC0`). An entry may additionally be bound to one specific key id, in which case the server compares the low four bits of the wire key selector against the entry's `key_id` and aborts with `FBSEC_ABORT_KEY_ID` (`0xC4`) on a mismatch. The sentinel `FBSEC_SOD_KEY_NONE` (`0x00`) means the entry accepts any provisioned key, and all four demonstration entries use it.

The second gate is policy. Two host hooks decide, in order:

- `fbsec_sod_port_access_allowed(op, data_id)` is the application-level gate for lock state and write protection. Returning false aborts with `FBSEC_ABORT_DEVICE_STATE` (`0x62`). The reference implementation allows everything.
- `fbsec_sod_port_role_allowed(op, key_id_base, data_id)` is the per-role gate. Returning false aborts with `FBSEC_ABORT_ROLE_DENIED` (`0xC3`).

The reference role policy is deliberately minimal: reads are always allowed, and writes are allowed only for key ids 1 and 2.

Key id	Role	Read	Write
1	Provisioning Key	yes	yes
2	Integrator Key	yes	yes
3	Operator Key	yes	no
4 to 15	Application-defined	yes	no

A secure write attempted under the Operator Key therefore passes the cryptographic check cleanly and is then refused at the policy layer with `0xC3`. That is a deliberate refusal, not a cryptographic failure. Note that key ids 4 through 15 are described as available for application use, but they are read-only under this policy until the hook is replaced.

NOTE: There is deliberately no asymmetric entry in this table. Every key id names a symmetric session key. The optional Ed25519 layer is a separate mechanism, not a fourth key tier: a signed access is guarded by an Ed25519 signature that replaces the AEAD tag, reached through the dedicated RPK objects rather than by a bit on a symmetric key selector. An object is guarded by AEAD or by a signature, never both. Identities are held per device rather than per slot, so there is no slot to select: a device signs with its own IDevID or LDevID, and a verifier resolves the peer by node id. Ed25519 exists to bootstrap and to sign for the symmetric key model, not to replace it, which is why the signed Provisioning-Key install at C02Fh delivers a symmetric key. Chapter 11 covers the layer.

Integrators replace the single function `fbsec_sod_port_role_allowed()` to install their own per-entry, per-role mapping. No change to the cryptographic code, the registry or the wire protocol is needed.

5
Wire Reference

This chapter fixes every byte the demonstrator puts on the wire. EmSA-WP-105 defines the shape of secure object access but leaves the primitive, the key and tag sizes and the associated-data layout to the implementer; this chapter records the choices SOFA made and the framing the CANopen FD variant wraps them in. Each fact appears here once. Later chapters refer back rather than restate.

5.1 Cryptographic Selections

The CANopen FD variant selects the following, all of them build-time defaults that appendix B lists with their ranges:

Item	Selection
AEAD primitive	AES-128-GCM, NIST FIPS 197 with SP 800-38D. AES-256-GCM is selectable at build time.
Session key size	16 bytes for AES-128-GCM, 32 bytes for AES-256-GCM (`FBSEC_AEAD_KEY_SIZE`)
Nonce size	12 bytes (`FBSEC_AEAD_NONCE_SIZE`)
Tag size	16 bytes, the full GCM tag. Truncation down to 4 remains configurable (`FBSEC_AEAD_TAG_LEN_BYTES`)
Random size	12 bytes contributed by each peer (`FBSEC_AEAD_RANDOM_SIZE`)
Maximum plaintext	32 bytes (`FBSEC_AEAD_MAX_PROTECTED`)
Key derivation	HKDF-SHA-256, RFC 5869
Protocol version	`0x01` (`FBSEC_AEAD_PROTOCOL_VERSION`), one field revision with no version history
Primitive id	`0x01` for AES-128-GCM, `0x02` for AES-256-GCM

What these primitives cost on a real microcontroller, and how the alternatives compare, is measured at cansecurity.net/resources/crypto-benchmarks. That is the place to look before changing the AEAD selection for a constrained target.

Throughout this chapter, R is the random size, N is the entry plaintext length and T is the tag size. At the default configuration R is 12, T is 16 and N is 4 or 16 for the demonstration entries.

5.2 Wire Key Selector Byte

One byte carries four fields. It rides only on client requests, always as the first payload byte, and the server never echoes it. The whole byte is bound into the associated data, so any attempt to strip encryption, flatten a cyclic arm, repurpose a reserved bit or reroute to a different key fails tag verification.

Bits	Field	Meaning
7	Encryption	0 = authenticate only, plaintext on the wire. 1 = encrypt and authenticate.
6	Cyclic arm	1 asks the server to keep a session alive after this access. Meaningful only on a read or write challenge; ignored on poll directions.
5	Reserved	Must be 0.
4	Reserved	Must be 0.
3 to 0	Base key selector	1 to 15. Selects the session key slot.

A non-zero reserved bit is rejected with `FBSEC_ABORT_KEY_ID` (`0xC4`), so a future bit assignment cannot collide silently with an older peer. The reserved mask covers bits 5 and 4. This byte belongs to the AEAD mechanism; the RPK objects are signed and carry no wire key selector.

The extraction macros in `shared/fbsec_aead.h` are `FBSEC_AEAD_KEYID_ENCRYPT`, `FBSEC_AEAD_KEYID_IS_CYCLIC`, `FBSEC_AEAD_KEYID_RESERVED` and `FBSEC_AEAD_KEYID_BASE`, with `FBSEC_AEAD_KEYID_MAX` set to 15.

Worked examples that appear in the bus trace:

Byte	Meaning
`0x82`	Encrypt, Integrator Key, single-shot
`0xC2`	Encrypt, Integrator Key, cyclic-capable single
`0x01`	Authenticate only, Provisioning Key, single-shot
`0xC1`	Authenticate only, Provisioning Key, cyclic-capable single

5.3 Direction Codes

The direction byte sits at offset 2 of the associated data and names the exchange step.

Code	Direction	Associated data built
`0x01`	READ_CHALLENGE	none
`0x02`	READ_RESPONSE	prefix plus both randoms
`0x03`	WRITE_CHALLENGE	none
`0x04`	WRITE_REQUEST	prefix plus both randoms
`0x05`	READ_POLL_REQUEST	prefix only
`0x06`	READ_POLL_RESPONSE	prefix only
`0x07`	WRITE_POLL_REQUEST	prefix only
`0x08` to `0xFF`	reserved	not assigned

The two challenge directions carry no tag, so no associated data is constructed for them.

5.4 Associated Data

The associated-data prefix is twelve bytes on the CANopen FD variant, where `FBSEC_AEAD_DEV_ID_SIZE` is 1 and CANopen node ids fit in a byte.

Offset	Size	Field
0	1	Protocol version, `0x01`
1	1	Mechanism byte: the encryption flag in bit 7 over the primitive id
2	1	Direction code
3	1	The full wire key selector byte
4	1	Server node id, the responder
5	1	Client node id, the requester
6 to 9	4	`data_id`, little-endian
10, 11	2	Data length, little-endian

Setting `FBSEC_AEAD_DEV_ID_SIZE` to 2 widens the prefix to fourteen bytes: the server device id moves to offsets 4 and 5 as a little-endian 16-bit value, the client device id to 6 and 7, the `data_id` to 8 through 11 and the length to 12 and 13. That width is reserved for future variants. Peers built with different widths see different prefixes and fail closed at the first secure verb, which is the intended outcome; the protocol version does not bump for this knob.

What follows the prefix depends on the direction:

- `READ_RESPONSE` and `WRITE_REQUEST` append `client_random[R]` then `server_random[R]`, client first in both cases.
- The three poll directions append nothing.
- In authenticate-only mode, that is when bit 7 of the key identifier is clear, the plaintext is appended after any randoms, because it is not carried as ciphertext and must still be covered by the tag.

5.5 Nonce Construction

The GCM nonce is uniform across single-shot and cyclic use:

```
nonce_base = client_random[0..11] XOR server_random[0..11]
nonce      = nonce_base XOR (0^64 || counter_be32)
```

For a single-shot transfer the counter is zero, so the nonce reduces to the exclusive-or of the two randoms. For a cyclic session both peers retain `nonce_base` for the life of the session, discard the original randoms and increment a 32-bit counter per poll. The counter is exclusive-ored into the last four bytes of the base.

The nonce stays unique and unpredictable as long as at least one peer draws a sound random. Exclusive-or with such a value carries the freshness even when the other side is weak, and only a simultaneous collapse of both random sources degrades the nonce space. This assumes both peers are honest; a peer that deliberately chooses its random to force a collision is a compromised endpoint and is out of scope. Both randoms are bound into the associated data, so an in-flight tamper of either contribution fails verification.

Poll frames also carry the low byte of the counter on the wire. That byte is a desync cross-check only. It is not part of the associated data, and it is not what makes the nonce unique.

5.6 Payload Shapes

Each flow is a short sequence of USDO exchanges. The byte sequences below are what the secure-tunnel layer hands to the transport.

Single secure read. Pass 1 is a download request carrying `key_id[1]` and `client_random[R]`, thirteen bytes at the default configuration, answered by an empty download response. Pass 2 is an empty upload request, answered by an upload response carrying `server_random[R]`, `cipher[N]` and `tag[T]`.

Single secure write. Pass 1 is an empty upload request, answered by an upload response carrying `server_random[R]`. Pass 2 is a download request carrying `key_id[1]`, `client_random[R]`, `cipher[N]` and `tag[T]`, answered by an empty download response.

Cyclic-capable single. Identical to the corresponding single, with bit 6 of the key identifier set. The server retains a session keyed by the client node and the `data_id`.

> **NOTE:** The data-bearing response of a cyclic-capable single is byte-identical to the single-shot response. No session identifier is carried on the wire. Both peers identify the session by the pair of client node and `data_id`, and bind freshness through the nonce.

Cyclic poll read. Both frames are a download exchange, not an upload one. The download request carries `counter_low[1]`, and the download response carries `counter_low[1]`, `cipher[N]` and `tag[T]`. A poll therefore costs one request and one response, against the two exchanges a single-shot read needs.

Cyclic poll write. The download request carries `counter_low[1]`, `cipher[N]` and `tag[T]`. The download response is empty and unauthenticated.

The shapes above are the AEAD verbs. A signed (RPK) access does not add a signature to these frames: it is a separate exchange through the dedicated RPK objects of chapter 4, where a 64-byte Ed25519 signature replaces the 16-byte AEAD tag. The signature is computed over a CiA 720-1 transcript: the domain string `CiA-720-V0.0`, then a `0x00` separator, the algorithm byte `0x30` (Ed25519), the role byte and the body. The body binds, in order, the responder and requester ids, the object multiplexor, the protected-data length, the requester random, the responder random and the value. Both directions are a two-pass challenge: the server issues a fresh random and the client answers with its own; on a signed read the device then returns the value and its signature, and on a signed write the client sends the value and its signature. The role byte is `0x05` for a signed read (server to client), `0x04` for a signed write (client to server) and `0x09` for a signed command.

5.7 USDO Transport

SOFA transports the byte sequences above in USDO (Universal Service Data Object) transfers as CiA-1301 defines them. The protocol data unit, the expedited and segmented command specifiers, the session handling and the reserved CAN identifiers are standard. Consult CiA-1301 for that detail; it is not repeated here.

Two points are specific to SOFA and are stated elsewhere in this manual:

- Which object an exchange addresses. The index and subindex are carried in the USDO header as usual, and chapter 4 gives the mapping to SOFA's `data_id`.

- What the data block contains. That is the Payload Shapes section above.

Aborts are standard too. An abort is the seven-byte frame of CiA-1301 Table 32, with command specifier `0x7F` and a single-byte code at offset 6. Appendix A lists which codes SOFA sends, including the `0xC0` block it assigns for authentication failures, which Table 31 does not cover.

Where USDO is not available, the same byte sequences fit inside classical CiA-301 SDO transfers. Each pass becomes one initiate-SDO plus zero or more segment-SDO transfers. The bytes in the SDO data fields are identical; only the framing differs.

5.8 Simulated Bus Carrier

The bus simulator is a TCP relay on `127.0.0.1:5810` that broadcasts every frame to all peers except the sender. It carries CAN FD frames in a six-byte header plus payload:

Offset	Size	Field
0 to 3	4	CAN identifier, 32-bit little-endian. Bit 31 IDE, bit 30 RTR, bit 29 ERR, bits 28 to 0 the identifier.
4	1	Flags: bit 0 BRS, bit 1 ESI
5	1	Payload length, 0 to 64
6 and up	N	Payload

The maximum wire frame is therefore 70 bytes. The relay accepts up to `FD_SETSIZE - 1` peers with a listen backlog of 16, and holds a named mutex so a second bus instance cannot start against the same port.

A peer announce carries the role byte, the node id and an optional name of up to 30 characters. The role byte is `0x01` for a server and `0x02` for a client.

When the relay rejects a frame it broadcasts a frame-error event carrying one of these codes:

Code	Reason
`0x01`	Reserved CAN identifier bits set
`0x02`	Reserved flag bits set
`0x03`	Payload length above 64
`0x04`	Short read on the TCP stream
`0x05`	Unknown CAN identifier
`0x06`	Malformed USDO frame, including a bad `data_id` form
`0x07`	Duplicate peer announce

The relay also drops the offending peer's connection after broadcasting the event, so a malformed frame costs the sender its session on the bus and not just that one frame. A well-formed frame addressed to a node that is not present is relayed normally and simply ignored; that is not an error.

5.9 Multi-Frame Bodies

Every symmetric verb fits in one expedited frame. The optional asymmetric layer does not: a 64-byte signature, the 40-byte identity blob, the 168-byte identity reply, the 108-byte voucher, the 80-byte provisioning-key install and the 96-byte LDevID export all exceed what one frame carries.

These use the CiA-1301 USDO segmented transfer protocol, unchanged. SOFA defines no fragmentation scheme of its own; moving a large body is the transport's job. Consult CiA-1301 for the segmented protocol.

Two properties of it are worth stating because they differ from classical CANopen SDO, and are easy to get wrong when porting:

- There is **no toggle bit**. Segments carry an 8-bit sequence counter that starts at zero and increments. A receiver that sees a gap aborts the transfer.
- Segment and end frames carry **no multiplexor**. The index and subindex appear only in the initiate frame, so a receiver binds the remaining frames to the transfer by peer and session.

> **NOTE:** SOFA's handover reads are a request that carries a challenge and expects a large reply, and CiA-1301 has no single service for that shape. The server therefore answers such a download request with an upload-initiate response, after which the exchange is ordinary segmented upload. This is the one profile-level liberty SOFA takes with the transport, and the codec header records it.

6
Building from Source

The package ships prebuilt executables, so running the demonstration needs no build. Build from source to change the configuration, to read the code with a debugger attached, or as the first step in porting to another target.

6.1 Default Build

Configure and build with CMake 3.20 or later and MSVC:

```
cmake -S . -B build -G "Visual Studio 18 2026" -A x64
cmake --build build --config Release
```

The three executables appear under `build/variants/canopen_fd/`, in the `bus/`, `server/` and `client/` subdirectories.

The project is C99 with compiler extensions off. Under MSVC it builds with `/W4 /WX /permissive-` and links the static C runtime, so the binaries carry no redistributable dependency.

6.2 Configuration

Chapter 5 states what the default configuration puts on the wire; appendix B lists every knob with its default and range. In summary, three groups of settings exist:

1. CMake cache variables, set with `-D` at configure time. These select the feature subset, the optional asymmetric layer and the peer-identifier width.
2. Macros edited in `shared/fbsec_config.h`. These select the AEAD primitive, the tag length, the encryption default, the random size and the per-key frame budget.
3. Registry limits in `shared/fbsec_secure_od.h`, such as the entry count, the key slot count and the two timeouts.

The AEAD algorithm and the tag length are deliberately not CMake variables. To build for AES-256-GCM, set `FBSEC_AEAD_AES256_GCM` to 1 and `FBSEC_AEAD_AES128_GCM` to 0 in the header; the session key becomes 32 bytes. To change the tag length, edit `FBSEC_AEAD_TAG_LEN_BYTES`, which accepts 4 through 16 for AES-GCM.

An inconsistent configuration fails at build time rather than at run time. Appendix B lists the assertions.

6.3 Building the Asymmetric Layer

The Ed25519 identity layer is built in by default, and so is the authorized handover model. The default configure line above therefore serves both the AEAD and the RPK blocks. To build a minimal AEAD-only device, turn the layer off at configure time:

```
cmake -S . -B build -DFBSEC_FEATURE_ASYM=0
```

`FBSEC_HANDOVER_AUTHORIZED` follows `FBSEC_FEATURE_ASYM` and defaults on with it. Set it to 0 for the basic, born-open model. Setting it to 1 while `FBSEC_FEATURE_ASYM` is 0 is a configure-time error.

NOTE: Building SOFA does not verify it. The distribution ships no test suite, and a clean build is evidence only that the code compiles. Qualifying a device built on SOFA needs a test campaign of its own, covering the cryptographic primitives against their published vectors, the wire protocol against CiA-1301, and the device's own key handling. EmSA offers that as a separate engagement.

6.4 Platform Support

The demonstrator is a Windows program. Two server port hooks are stubs outside Windows: `fbsec_sod_port_get_time_ms()` returns 0 and `fbsec_sod_port_random()` returns false. With a time source stuck at zero the challenge timeout and the cyclic idle timeout do not work, and with the random hook failing every secure verb aborts with `FBSEC_ABORT_INTERNAL` (`0x60`).

Porting to another host therefore starts by implementing those two hooks, plus the client's random hook. Chapter 9 lists the full hook surface.

7
Running the Demo

The demonstration needs three windows: the simulated bus, a server and a client. Start them in that order. Chapter 8 is the full option reference; this chapter is the walkthrough.

7.1 Starting the Three Programs

Three launcher batches live under `run/`:

1. `start_fd_hub.bat` starts the simulated CAN FD bus. It listens on TCP `127.0.0.1:5810` and optionally bridges to a PEAK CAN FD interface through the auto-detect prompt at start-up.
2. `start_fd_server.bat` starts a CANopen FD server, loading session keys from `run/keys-demo.txt` and the constant object-dictionary entries, object 1018h among them, from `run/sofa-od-demo.txt`. It prompts for the node id, defaulting to `0x05`.
3. `start_fd_client.bat` starts the client in interactive menu mode, also loading `run/keys-demo.txt`.

At start-up the client prints how many session keys it loaded, then prompts in turn for its own node id, the target node id and the encryption mode. The key choice is deferred, because an uncommissioned device has no key to use until a handover installs one: the client asks which key to use the first time a keyed operation runs, and once more right after the lifecycle submenu finishes commissioning and assigns the keys. The choice defaults to the Integrator Key, and a key fixed on the command line skips the prompt.

7.2 The Menu

A scan action, the numbered options and a quit key are offered. Options 1 through 6 are the AEAD verbs and appear in every build. Options 7 through 10 are the RPK verbs and appear only in an asymmetric build:

Choice	Action	Address	Payload written
A	Scan security parameters	C000/C001/ C011/1018	none, no key
1	Single 16-byte secure read	`0xC01800`	none
2	Single 16-byte secure write	`0x201600`	a 32-bit big-endian counter starting at 1, then twelve `0xCC` bytes
3	Single 4-byte secure read	`0x202000`	none
4	Single 4-byte secure write	`0x201000`	`12 34` then a 16-bit big-endian counter starting at 0
5	300 cyclic 4-byte reads at 200 ms	`0x202000`	none
6	300 cyclic 4-byte writes at 200 ms	`0x201000`	the poll index as a 32-bit big-endian counter, `00 00 00 01` upward. Not the option 4 pattern.
7	Signed identity read (C028h)	`0xC02800`	a 16-byte challenge

Choice	Action	Address	Payload written
8	Signed read of 0x2021 (C042h)	`0xC04201`	none
9	Signed write of 0x2017 (C042h)	`0xC04202`	a 16-byte demonstration value
10	Signed command (C049h)	`0xC04900`	a 32-bit command code
Q	Quit the client		

Option 1 reads the C018h authenticated identification, which returns the object 1018h identity quad under an AEAD tag. Option 2 writes the 16-byte demonstration entry at 2016h.

The four RPK options exercise the signature-guarded objects of chapter 4, where an Ed25519 signature replaces the AEAD tag. Option 7 asks the device to prove genuineness under its IDevID. Options 8 and 9 read and write the RPK twins 0x2021 and 0x2017 through the signed generic access object C042h, the same read and write shown for AEAD in options 3 and 2 but under a signature. Option 10 sends a signed function command through C049h.

Scanning Security Parameters

Action `A` is a cold, unsecured sweep of the constant security parameters, run before any key is selected. It reads every subindex of C000h, C001h and C011h, and object 1018h, and appends each value's symbolic meaning to its trace line: the profile and AEAD suite from C000h, the commissioning state and installed keys from C001h, the per-slot key ids from C011h, and the vendor id, product code, revision and serial from 1018h. An abbreviated run against a node-5 server reads:

```
--- security parameter scan (target 0x05, no key) ---
  C000h capabilities:
    sub 0x00                         highest sub-index 0x08
    sub 0x01  00 00 00 00            type word: profile Custom, level C0
    sub 0x02  01 10 00 00            AES-128-GCM, tag 16 bytes
    sub 0x03  01 00                  KDF: HKDF-SHA-256
    sub 0x04  01 00                  signature: Ed25519
    sub 0x05  07                     symmetric key levels: Provisioning Integrator Operator
    sub 0x06  01                     asymmetric key presence: IDevID
    sub 0x07  05                     claim gates: TOFU voucher
    sub 0x08  03 00                  mechanisms: AEAD RPK
  C001h status:
    sub 0x00                         highest sub-index 0x05
    sub 0x01  01                     commissioning: commissioned/owned
    sub 0x02  04                     active claim gate: voucher
    sub 0x03  07                     keys installed: Provisioning Integrator Operator
    sub 0x04  01 10 00 00            active AEAD: AES-128-GCM, tag 16 bytes
    sub 0x05  01                     device identities: IDevID
  C011h AEAD key ids:
    sub 0x00                         highest sub-index 0x04
    sub 0x01  01 10 00 00            Provisioning key id = 0x00001001
    sub 0x02  02 10 00 00            Integrator key id = 0x00001002
    sub 0x03  03 10 00 00            Operator key id = 0x00001003
    sub 0x04  00 00 00 00            Application key: empty slot
  1018h identity:
    sub 0x00                         highest sub-index 0x04
```

```
    sub 0x01  00 00 02 F1              Vendor ID = 0x000002F1
    sub 0x02  53 4F 46 41              Product code = 0x534F4641 ("SOFA")
    sub 0x03  00 01 00 01              Revision = 0x00010001
    sub 0x04  00 00 00 01              Serial number = 0x00000001
```

The C000h and C011h U32 values travel little-endian, so the wire bytes read reversed from the decoded value on the same line. Object 1018h is served big-endian, as authored.

Options 5 and 6 run one arming access followed by 299 polls against the same session. The counters in options 2 and 4 advance on every invocation, so repeating an option shows a different payload each time. This is what makes the auto-incrementing behavior of `0x2020` and the shadowing between `0x2010` and `0x2020` visible from the menu alone: write with option 4, then read with option 3, and the value just written comes back.

Option 5 makes the auto-increment visible directly: each of its 300 reads returns a value one higher than the last. Because a run is longer than 256 frames, it also shows the poll counter byte on the wire wrapping through `0xFF` back to `0x00` at poll 257 while the session continues normally. That byte is the low byte of a 32-bit counter, and chapter 5 explains why only the full counter matters.

The addresses shown in the menu are the 24-bit form. On the command line the full 32-bit form is required instead; chapter 4 explains the reserved low byte and chapter 8 gives the command-line rule.

NOTE: Whatever the menu asks for, per-key frame use is hard-capped at 1 000 000 frames. Reaching the cap tears the cyclic session down and the client must arm a fresh one. At the menu's 300-frame runs the cap is never approached.

7.3 Key File Format

The server and the client both load session keys from a text file given with `--key-file`. A single key can also be passed inline; chapter 10 explains the difference between `--key` and `--main-key`.

The file holds one row per key. Blank lines and lines starting with a hash are ignored. Each row is the key id, a free-form role label, the key as hex bytes and an optional non-secret id, separated by whitespace:

```
# keyid  role-label      key (hex)                           id
1        prov-session    00112233445566778899AABBCCDDEEFF    0x1001
```

The key id runs from 1 to 15 on the client and from 1 to 4 on the server, as chapter 4 explains. The role label is informational, because the parser keys on the id. The hex value must be exactly the session key length: 16 bytes for AES-128-GCM or 32 bytes for AES-256-GCM. Separators inside the hex value are ignored, so spaces, colons, hyphens and commas may be used for readability. The fourth column is the optional U32 key id or version the server reports through C011h; when it is absent the id defaults to the key id. The column is read by the server and ignored by the client, which needs only the key material.

CAUTION: The server loads the key file first, treats each slot as write-once, and then backfills the public demonstration keys into any slot the file left empty. A key file that provisions only some slots therefore leaves demonstration keys live in the rest. Appendix C prints those keys.

7.4 Optional CAN FD Bridge

With no arguments the bus probes for a PCAN device at start-up. If a CAN FD-capable channel is found in the AVAILABLE or PCANVIEW state, the bus prints a prompt of the form:

```
PCAN device on PCAN_USBBUS1 detected. Connect at 500k/2M? (y/n):
```

Answering `y` bridges every USDO frame seen on the simulated bus onto the real CAN FD network at 500 kbit/s arbitration and 2 Mbit/s data, and broadcasts every CAN FD frame received from the real bus back to all simulated peers. A hardware CANopen FD server or client can then join the simulated mesh on equal footing.

Simulator-internal control frames, meaning peer announce, peer loss and frame error, are not bridged. They have no meaning on a physical bus. Chapter 5 lists their CAN identifiers.

If the detected device is classical-CAN only, the bus reports that no detected device supports CAN FD and continues in TCP-loopback mode. SOFA USDO frames carry up to 64 payload bytes and are too long for classical CAN.

7.5 Reading the Trace

Each program emits a color-coded trace on standard output. The direction tints are blue for requests, meaning transmit at the client and receive at the server, magenta for responses, red for aborts, and gray for metadata such as peer announce and loss or acknowledgement envelopes.

The bus trace columns are the frame number, the timestamp, the source and destination node id pair in two-digit hex, the CAN identifier, and the payload in hex.

The client and server per-request traces use a similar layout: the timestamp, the direction, the verb tag, the source and destination node ids, the `data_id` in hex, and the payload in hex. On the receiving side a `Plain:` line follows, showing the decrypted plaintext.

The server labels each inbound frame with the pass or object it recognized: `SRD1` and `SRD2` for the two passes of a secure read, `SWR1` and `SWR2` for a secure write, and `SRD?` or `SWR?` when the `data_id` is not registered. The cold, unsecured objects have their own tags: `CAP` and `STAT` for the C000h and C001h descriptors, `CRD` for a constant read such as object 1018h, and `C010`, `C011` and `C01F` for the AEAD-block objects the security handler serves. Asymmetric builds add `HOID`, `HOVC`, `HOEP`, `HOPK` and `HOLD` for the handover objects.

The first byte after the verb tag in every client request that carries one is the wire key identifier. Chapter 5 decodes it and lists the values that appear in the demonstration.

When a commissioning step changes the device state, the server reprints the status lines it first showed at start-up: the `key store` summary when a key was installed, and the commissioning and handover-gate lines when the lifecycle stage advanced. The console therefore tracks the current ownership and key state without a restart. The `key store` summary names the three base keys, Provisioning, Integrator and Operator, and reports each as loaded or absent.

7.6 Shutting Down

Press `Q` in the client menu to quit the client, then Ctrl-C in the server and bus windows. The bus uninitializes any open PCAN channel automatically on Ctrl-C, so no manual cleanup is needed.

The bus holds a named single-instance mutex and retries the bind on a port already in use, so restarting quickly after a shutdown is safe.

8

Command-Line Reference

The demonstrator ships three programs. This chapter lists every option each one accepts, its default and the rules that govern how options combine. None of the three reads an environment variable; everything is passed on the command line or comes from a key file.

All three print the EmSA banner before any operational output, and all three accept `--help`, which prints usage and exits without starting anything.

8.1 Bus Simulator

The bus simulator relays CAN FD frames between the connected peers and, optionally, a real CAN FD network. Invoke it as `fbsec_co_fd_bus [options]`.

Option	Default	Meaning
`--port N`	5810	TCP port to listen on.
`--pcan CHANNEL`	auto-probe	Bridge non-interactively to the named PEAK channel. Accepts `PCAN_USBBUS1` through `PCAN_USBBUS8`, `PCAN_PCIBUS1` through `PCAN_PCIBUS8`, or a hex literal such as `0x51`.
`--no-pcan`	off	Skip the auto-probe prompt entirely.
`--color`, `--no-color`	auto	Force ANSI color on or off.
`--quiet`	off	Suppress the trace on standard output.
`--help`		Print usage and exit.

With no PEAK option the bus probes for a device at start-up and prompts before bridging. The bridge runs at 500 kbit/s arbitration and 2 Mbit/s data; that pair is fixed.

The channel handles behind the symbolic names are `0x51` through `0x58` for `PCAN_USBBUS1` to `PCAN_USBBUS8` and `0x41` through `0x48` for `PCAN_PCIBUS1` to `PCAN_PCIBUS8`.

8.2 Server

Invoke the server as `fbsec_co_fd_server [options]`.

Option	Default	Meaning
`--bus HOST:PORT`	127.0.0.1:5810	Bus simulator endpoint. A bare number is read as a port, a bare string as a host.
`--node N`	`0x05`	CANopen node id, 1 to 127. Mirrored into the associated-data device id.
`--name STR`	`fbsec_co_fd_server`	Name announced to other peers, up to 30 characters.
`--device-id N`	from `--node`	Override the associated-data device id. Must be `0x0001` to `0xFFFE`; `0x0000` is reserved and `0xFFFF` is broadcast.

Option	Default	Meaning
`--key-file FILE`	none	Load session keys from a key file. An optional fourth column per row sets the non-secret key id C011h reports. See chapter 10 for the format.
`--od-file FILE`	none	Load the constant, unsecured object-dictionary entries from a file, object 1018h among them. Without it, object 1018h is absent and C018h aborts. Chapter 4 gives the file format.
`--verbose`	off	Extra per-request detail in the trace.
`--quiet`	off	Suppress the trace.
`--color`, `--no-color`	auto	Force ANSI color on or off.
`--help`		Print usage and exit.

8.3 Client

The client runs either one verb from the command line or an interactive menu:

```
fbsec_co_fd_client (srd | swr | srdpoll | swrpoll) <target_node> <data_id> [options]
fbsec_co_fd_client --menu [options]
fbsec_co_fd_client --help
```

The four verbs are secure read, secure write, cyclic poll read and cyclic poll write. The plain `rd` and `wr` verbs exist in the shared client code but are rejected by the CANopen FD variant, which offers no plain access to application data. The capability and status descriptors of chapter 4 are plain read-only objects, but the client reaches them through the `caps` subcommand rather than through `rd`.

<target_node> is a node id from 1 to 127, in decimal or hex. <data_id> is the full 32-bit address, `0xIIIISS00`, where the index must be `0x1000` or above and the low byte must be `0x00`. The 24-bit form that traces and the menu display is not accepted here, so write `0xC0180000` rather than `0xC01800`.

Transport and identity options:

Option	Default	Meaning
`--bus HOST:PORT`	127.0.0.1:5810	Bus simulator endpoint.
`--node N`	`0x7F`	Own CANopen node id, 1 to 127. Must differ from the target.
`--target-node N`	`0x05`	Target node id used by the menu and as the default for a verb.
`--name STR`	`fbsec_co_fd_client`	Name announced to other peers, up to 30 characters.
`--device-id N`	from `--node`	Override the associated-data device id, `0x0001` to `0xFFFE`.

Key options:

Option	Default	Meaning
`--keyid N`	none	Select the key slot, 1 to 15. Must appear before `--key`.

Option	Default	Meaning
`--key HEX`	none	Install an already-derived session key into the slot named by `--keyid`.
`--main-key HEX`	none	Install a base key from which the session key is derived. Requires `--salt`.
`--salt HEX`	none	Derivation salt, 1 to 32 bytes.
`--kdf NAME`	`FBSEC-SK-v1`	Key-derivation label. Only the literal `FBSEC-SK-v1` is accepted.
`--key-file FILE`	none	Load session keys from a key file.

Behavior options. Several of these are accepted by the shared client parser but are not listed by `--help`, which prints an abridged summary. The table below is the full set:

Option	Default	Meaning
`--data HEX`	none	Payload for a write verb, 1 to 32 bytes. Required for `swr` and `swrpoll`.
`--count N`	1	Number of iterations. Non-zero.
`--timeout MS`	1000	Response timeout in milliseconds. Non-zero.
`--encrypt`, `--no-encrypt`	encrypt	Set or clear bit 7 of the wire key selector.
`--stop-on-fail`	off	Stop a repeated run at the first failure.
`--menu`	off	Start the interactive menu.
`--hex`	off	Render payloads as hex only.
`--batch FILE`	none	Read commands from a file.
`--out FILE`	none	Write the trace to a file as well.
`--timestamp`, `--no-timestamp`	on	Show or hide trace timestamps.
`--verbose`	off	Extra per-request detail.
`--quiet`	off	Suppress the trace.
`--color`, `--no-color`	auto	Force ANSI color on or off.
`--help`		Print usage and exit.

Four rules govern how the key options combine, and the client refuses to start if any is broken:

1. `--keyid` must appear before `--key` on the command line, because `--key` installs into the slot most recently named.
2. `--key` and `--main-key` are mutually exclusive. One installs a session key directly, the other derives one.
3. `--main-key` requires `--salt`.
4. A secure verb needs a key id plus either `--key` or the pair `--main-key` and `--salt`, or else a `--key-file`.

The client's own node id must differ from the target node id.

Two subcommands bypass the verb parser entirely. `caps <node>` reads and decodes the capability descriptor of chapter 4; it needs no key, because the record is unauthenticated, and it is built into every configuration. `commission <node>` runs the handover flow of chapter 11 and exists only in asymmetric builds. Both accept `--bus` and `--node`:

```
fbsec_co_fd_client caps <target_node> [--bus HOST:PORT] [--node N]
fbsec_co_fd_client commission <target_node> [--bus HOST:PORT] [--node N]
```

8.4 Exit Codes

Every client invocation maps its outcome onto one of four process exit codes, so a script can distinguish a local failure from a server refusal and from a cryptographic failure.

Exit	Meaning
0	Success.
1	Local error: timeout, transmit failure, malformed reply, buffer too small, or a random-source failure.
2	The server returned an abort. Appendix A lists the codes.
3	AEAD tag verification failed locally.

CAUTION: Exit code 1 covers a random-source failure. Treat that case as "the random number generator is broken, do not continue" rather than as a transient error worth retrying.

8.5 Launcher Batches

Three batch files under `run/` wrap the common invocations. Each takes optional positional arguments.

Batch	Arguments	Effect
`start_fd_hub.bat`	port, default 5810	Starts the bus simulator.
`start_fd_server.bat`	node then port, defaults `0x05` and 5810	Starts a server with `--key-file run/keys-demo.txt` and `--od-file run/sofa-od-demo.txt`. Prompts for the node id if it is not given.
`start_fd_client.bat`	port then target node, defaults 5810 and `0x05`	Starts the client in `--menu` mode with `--timeout 1500` and `--key-file run/keys-demo.txt`.

9

Integration Guide

This chapter is the porting guide for engineers taking SOFA off the Windows demonstrator and onto a real CANopen FD device. It covers the object-dictionary surface: which hooks the secure-object layer calls into the host, which entry points the host calls into it, where secure entries belong in a device's object dictionary, and how the two-pass flow maps onto USDO. Chapter 10 covers the other surface, key handling.

The cryptographic contract is out of scope here and is fixed in chapter 5. An integrator does not reimplement the associated-data construction, the nonce derivation or the key identifier semantics; those are what every SOFA variant must honor unchanged.

9.1 Port Hooks

The server-side dispatch in `shared/fbsec_secure_od.c` calls seven hooks. The variant adapter never touches the AEAD or the registry bookkeeping directly.

Hook	Contract
`fbsec_sod_port_get_device_id()`	Returns the local CANopen node id, 1 to 127.
`fbsec_sod_port_get_time_ms()`	Millisecond clock, used for the challenge timeout and the cyclic idle timeout.
`fbsec_sod_port_random(buf, len)`	Fills `buf` with `len` bytes of cryptographic-quality random for `server_random`. Returning false surfaces as `FBSEC_ABORT_INTERNAL` (`0x60`).
`fbsec_sod_port_access_allowed(op, data_id)`	Application-level access gate for lock state and write protection. Returning false aborts with `FBSEC_ABORT_DEVICE_STATE` (`0x62`).
`fbsec_sod_port_role_allowed(op, key_id_base, data_id)`	Per-role policy. Returning false aborts with `FBSEC_ABORT_ROLE_DENIED` (`0xC3`). The `op` argument is `FBSEC_SOD_OP_READ` (0) or `FBSEC_SOD_OP_WRITE` (1).
`fbsec_sod_port_read_before(data_id, dst, len)`	Fills `dst` with the plaintext for a read-only entry just before the AEAD seal. `*len` is the maximum on entry and the actual length on return. Returns 0 or an abort code.
`fbsec_sod_port_write_after(data_id, src, len)`	Commits `len` verified plaintext bytes for a write-only entry just after the AEAD open. Returns 0 or an abort code.

The client side requires one hook, `fbsec_secure_port_random(buf, len)`, declared in `shared/fbsec_secure_proto.h`. It fills `buf` with `client_random` for each secure verb under the same contract as the server random hook.

Asymmetric builds add two hooks per side. The server implements `fbsec_sod_port_sign()` to sign with the device identity and `fbsec_sod_port_peer_pubkey()` to fetch a peer identity to verify against; the client implements the mirror pair `fbsec_secure_port_sign()` and `fbsec_secure_port_peer_pubkey()`.

9.2 Integrator Entry Points

Where the hooks are functions the secure-object layer calls into the host, these are what the host calls into it. They are declared in `shared/fbsec_secure_od.h` and `shared/fbsec_hkdf.h`.

Function	Purpose
`fbsec_sod_init()`	Resets the registry, the key slots and the challenge state.
`fbsec_sod_register_entry()`	Registers a secure entry. The `fbsec_sod_entry_t` struct carries `data_id`, `key_id`, `access_flags`, `value_type` and `data_len`.
`fbsec_sod_set_key()`	Installs a session key into a slot. Refuses an already-populated slot.
`fbsec_sod_dispatch()`	Handles one inbound request and fills either the reply payload or `out_abort`.
`fbsec_hkdf_sha256()`	Derives a session key, for hosts that derive rather than install directly.

An entry's `key_id` is 1 to 15, or `FBSEC_SOD_KEY_NONE` to accept any provisioned key. Its `access_flags` are `FBSEC_SOD_ACCESS_SECURE_RO`, `FBSEC_SOD_ACCESS_SECURE_WO`, or both bits for an entry that is readable and writable. The demonstration entries are pure read-only or pure write-only for clarity.

9.3 Object Dictionary Entry Placement

Place secure entries in the manufacturer-specific area of the CANopen object dictionary, which CiA-1301 section 7.4.7 gives as indices `0x2000` through `0x5FFF`. Within that range the integrator picks any layout that fits the device's existing dictionary.

The demonstrator uses `0x2010` and `0x2020` for a paired 4-byte read and write, and `0x2016` for a 16-byte write target. These application entries are not normative; an integrator picks any manufacturer-specific layout. The identity object C018h is different: it is the CiA 720 authenticated-identification object and sits in the reserved security range, not the manufacturer area. What matters for the application entries is that each index and subindex pair the integrator calls secure is registered with `fbsec_sod_register_entry()`, and that the matching CANopen descriptor marks the entry with the right access flags for the device profile.

A plain, non-secure read or write against a secure entry must be rejected by the object-dictionary access layer. That rejection sits below the secure tunnel and is separate from the abort codes in appendix A. On a CANopen FD stack the USDO codes `0x32` for writing a read-only object and `0x31` for reading a write-only one fit the two directions.

Entry data length is fixed per entry. On every read the read-before callback must return exactly `data_len` bytes, and on every write the request must carry exactly that many bytes. Variable-length entries are out of scope in this revision.

9.4 USDO Transport Mapping

Chapter 5 gives the byte sequences that go in the USDO data block; the USDO protocol itself is standard CiA-1301 and an existing stack already implements it. What an integrator wires up is only the path from a USDO callback into `fbsec_sod_dispatch()` and back:

1. Validate that the access arrived over a USDO or SDO transport, not a PDO or a vendor-specific service.
2. Hand the data block into `fbsec_sod_dispatch()`.

3. Send the dispatch's reply back over the same USDO exchange, or encode `out_abort` into an abort frame.

The secure-object layer calls read-before and write-after at the right moment. The integrator does not call them directly.

`fbsec_sod_dispatch()` is single-threaded: one client request, one reply. If the host stack is itself multi-threaded, serialize calls into the dispatch with a mutex, and make the read-before and write-after callbacks do their own locking against device state.

9.5 What Integrators Do Not Wire

Three things are handled entirely inside the library and must not be reimplemented:

- The AEAD itself, meaning AES-GCM, HKDF, the associated-data prefix layout and the nonce derivation.
- Key selection. Provisioned session keys go in once through `fbsec_sod_set_key()`, and the secure-object layer picks the right one per incoming key identifier byte.
- Cyclic session bookkeeping. The layer keeps a slot per entry and direction, tracks the 32-bit frame counter that feeds the nonce, and tears the session down on timeout or on budget exhaustion.

10

Key Management

This chapter covers the second integration surface: where keys come from, how they reach the device, where they live on an MCU, and when they are rotated. Chapter 4 states the policy the reference server enforces once the keys are in place.

Key management is a subject in its own right, and this chapter only covers what SOFA needs. For the background, see cansecurity.net/key-management, and in particular symmetric-key-management for the per-device key model that the roles below assume.

10.1 Base Keys and Session Keys

On a real target, WP-104 specifies that each layer holds a base key, its Provisioning, Integrator or Operator Key, and that a per-session key is derived per communication through HKDF over that base key, a salt and an info string. The demonstrator simulates both the base keys and the derivation step out: what the command line loads, what `fbsec_sod_set_key()` installs and what the wire key selector selects are the already-derived session keys.

SOFA nonetheless ships the derivation helper for hosts that prefer not to store final session keys. The derivation is HKDF-SHA-256 with the base key as input keying material, a host-set salt, and an info string, producing an output of the session key length.

The info string is exactly thirteen bytes:

```
46 42 53 45 43 2D 53 4B 2D 76 31 00 <keyid>
"F  B  S  E  C  -  S  K  -  v  1"  NUL
```

Both peers that derive keys must use the same info string and salt. A mismatch surfaces cleanly as a tag-verify failure on the first secure verb.

> **NOTE:** The trailing `NUL` is part of the info string. The eleven-character literal is copied with a length of twelve, so the terminator lands in the derivation input before the key identifier byte. Any independent implementation must reproduce all thirteen bytes, not the eleven visible characters.

10.2 Two Install Patterns

Two patterns exist at the API level, and the command line mirrors them:

Direct session-key install. Host code calls `fbsec_sod_set_key(keyid, key)` once at boot per role, with the fully derived session key bytes. HKDF is bypassed. This suits keys stored in a per-device secure store, injected at production, or pushed by a provisioning tool that already did the derivation off-device. On the command line this is `--keyid N` followed by `--key HEX`.

Base key plus salt. Host code holds the base key, often in one-time-programmable memory or a secure element, reads a salt from operator-supplied configuration at boot, calls `fbsec_hkdf_sha256()` to derive the session key per role, and then `fbsec_sod_set_key()` to install it. This suits a base key provisioned once, with per-deployment salts rotating the session key without re-touching it. On the command line this is `--main-key HEX` together with `--salt HEX`.

Both sides treat each key slot as write-once per process. `fbsec_sod_set_key()` refuses to overwrite a populated slot. To rotate, tear the process or session down and provision again.

10.3 Provisioning Workflows

Three patterns cover most fielded cases. Pick one or combine them.

Factory inject. Per-device session keys are programmed at production time into the device's persistent store. On boot the firmware reads them back and installs one per role. This is the simplest model and leaves no long-term key to leak. The cost is that keys must be uniquely generated and tracked per device, and a field re-key needs a physical operation or a separately secured channel that itself uses another key. It is often the right pattern for small, low-rotation deployments.

Derive from the base key. Each device holds its per-role base keys, and session keys are derived at boot from the base key, the device id and a salt. Only the base keys ever leave the factory floor, and the salt rotates independently as the deployment toolchain pushes a new one each maintenance window. The persistent footprint per device is small and session keys rotate without re-provisioning. The trade is that a base-key compromise rotates every session key derived from it.

Scheduled re-key. Session keys are swapped on a planned cadence. The device tears its session down, re-runs whichever provisioning path applies, and resumes. Because slots are write-once per process, a session teardown is the natural rotation boundary.

Hot reload, meaning a key swap without a teardown, is deliberately not implemented. Rotating a key while a cyclic session is in flight risks tag forgery on the first frame after the swap. A deployment that needs sub-second rotation should model it as a re-arm: tear the cyclic session down, install the new key, arm a fresh session.

10.4 MCU Key Storage

The port-hook surface deliberately does not mandate where keys live. The host owns that decision. Five topologies are common. Which of them a given part supports is a procurement question; cansecurity.net/resources/secure-can-mcus tabulates the hardware security features of current CAN FD microcontrollers.

Volatile SRAM only. Keys are derived or fetched at boot and held for the process lifetime. A power cycle wipes them and provisioning repeats on the next boot. Use this when keys come from a hardware root of trust at boot, such as a PUF feeding HKDF, or when the device is paired to an attested controller that pushes keys after each reset. Debug-port or DMA readback exposure during the live session remains the host's problem; SOFA keeps the keys in a private static array inside `shared/fbsec_secure_od.c`.

Internal MCU flash. Keys live in a dedicated flash region read at boot. This is the cheapest persistent option. Enabling the part's readout protection is mandatory, so that a JTAG or SWD attacker cannot dump the page, and on parts that can fall back to a factory-reset state the anti-rollback fuse should be set as well. A single bad write cycle can corrupt the slot, so treat the region as a small append-only store with a CRC and a generation counter, or keep a known-good boot key alongside the active one.

One-time-programmable fuses. Truly write-once and factory-set. This suits a base key in the derive-from-base-key pattern: provision once at production and derive everything else at boot. Capacity is small, often one to four keys, and a base-key leak is permanent. Do not put session keys you intend to rotate here.

External secure element. A separate die such as an ATECC608, OPTIGA, TA100 or ST33, reached over I2C or SPI. The key never leaves the chip, and the application asks the element to perform the AEAD operation. This is the strongest topology for high-assurance deployments. The integration cost is that the AEAD shim in `shared/fbsec_aead.c` becomes a remote call into the element's command set. Replace the body of the seal and open functions; keep the associated-data construction and the header API intact so callers do not change.

TrustZone-isolated SRAM. On Cortex-M23, Cortex-M33 and similar parts, keys live in a Secure-world region and the Non-secure application calls a secure-side service for every AEAD operation. This is the practical middle ground when the silicon has TrustZone but no separate secure element. The HKDF implementation should live in the secure world too if session keys are derived at boot.

10.5 Random Number Requirements

Both peers consume randomness, twelve bytes each per single-shot transfer:

- The server issues a fresh `server_random` on every read challenge response and every write challenge. The Windows reference implementation uses the platform random API.
- The client generates a fresh `client_random` for Pass 1 of every read and Pass 2 of every write.

Chapter 5 gives the nonce construction that combines them.

> **CAUTION:** Both peers must use a cryptographic-quality random source. The mutual-random construction tolerates a low-entropy but honest source on one side while the other stays sound. If entropy fails on both peers the nonce space collapses and the tunnel breaks under a motivated attacker. Treat a random-source failure as fatal and stop, rather than continuing with predictable nonces.

A source seeded from wall-clock time is not adequate. On MCU targets without a hardware random generator, seed a DRBG (Deterministic Random Bit Generator) from the most physical entropy available, such as the low bits of an ADC or oscillator drift, and call the DRBG from the port hooks. A random failure surfaces as `FBSEC_ABORT_INTERNAL` (`0x60`) on the server and as exit code 1 on the client.

10.6 Key Rotation Policy

Single-shot verbs. Each invocation generates its own freshness, and there is no per-frame counter to overflow. Rotate on whatever schedule operations policy allows; the cryptography places no hard ceiling. NIST SP 800-38D limits AES-GCM to roughly 2^32 encryptions per key and initialization-vector prefix, but the mutual-random construction reaches that bound only if both peers' random sources collapse to a 96-bit collision space at the same time.

Cyclic sessions. The per-key, per-session budget is a hard wall at 1 000 000 frames. When the counter reaches it, both peers refuse further frames: the server tears the slot down and aborts with `FBSEC_ABORT_KEY_BUDGET` (`0xC7`), and the client returns a protocol error. Sessions also expire after 60 seconds without traffic, in which case the server drops the session silently and the next poll fails the same way. Recovery from either is a fresh cyclic-capable single access.

One million rather than 2^32 is a deliberate margin. Staying close to the NIST bound erodes the cryptographic margin, and a budget that low makes a key refresh a normal operational event rather than a once-in-a-device-lifetime ceremony. With the default full 16-byte tag the forgery margin is not the binding constraint at all; the budget matters most to a deployment that has truncated the tag to buy frame space. At a 1 ms poll period the budget is about 16 minutes; at 100 ms it is about 28 hours. Either way the operator sets a re-arm cadence well below the wall. Reaching the limit is not a failure. It is the protocol saying that it is time to rotate.

> **CAUTION:** AES-GCM breaks if the same nonce is reused under one key. A repeated key and nonce pair can expose the authentication key and voids both confidentiality and authenticity, so nonce uniqueness is not optional. Never replay a frame or roll a counter back.

11

Optional Asymmetric Identity

The asymmetric layer adds Ed25519 device identity beside the symmetric tunnel: signed device identification, signed generic access and a signed function command, together with a manufacturer-to-integrator handover. It is built in by default. Setting `FBSEC_FEATURE_ASYM` to 0 compiles it out, and the wire behavior is then byte-identical to a symmetric-only build.

Whether a deployment needs this layer at all is a design question that precedes SOFA. cansecurity.net/key-management/symmetric-vs-asymmetric sets out the trade, and /key-management/handover covers the manufacturer, integrator and operator model that the flow below implements.

Chapter 4 fixes the object addresses and payload sizes, chapter 5 the wire framing and segmented transfer, chapter 6 the build options and appendix C the demonstration identities. This chapter describes what the layer does with them. The normative model lives in EmSA-WP-104 and EmSA-WP-105.

11.1 Device Identities

Following IEEE 802.1AR, a device holds two Ed25519 identities:

- The **IDevID** is the factory identity. It is permanent, certified by the manufacturer, and used to prove genuineness.
- The **LDevID** is the local identity. It is generated on the device during handover and cleared by a manufacturer reset.

In the authorized model the device additionally holds the manufacturer trust anchor, a bare public key, which verifies ownership vouchers.

11.2 Capability and Status Descriptors

The C000h and C001h descriptors described in chapter 4 are the entry point to the layer. A client reads them cold, before any key or identity exists, to learn what the device supports.

Both records are present in every build with their full subindex range, and the `caps` subcommand that reads C000h is present in every build. In the default asymmetric build C000h advertises the layer: the mechanisms bitmap at C000h:08h sets the RPK bit, the RPK-algorithm subindex reads Ed25519, the identity flags report the IDevID (the LDevID appears once it is exported), and the handover model reads TOFU together with the manufacturer voucher. A minimal AEAD-only build clears all four, so the same read tells a client at once whether the device offers the layer.

A client reads a descriptor with `fbsec_client_read_descriptor()`, or through the `caps` subcommand, and fails closed on what the descriptor advertises. Chapter 4 explains why a spoofed descriptor cannot force a peer into a weaker mode. Because C000h advertises RPK in an asymmetric build, the layer is discoverable from the descriptor, and the `commission` subcommand drives the RPK objects.

11.3 Signed Access

A signed access is guarded by an Ed25519 signature that replaces the AEAD tag. It is not an extra trailer on an AEAD frame: an object is protected either by AEAD or by a signature, never both. The signed accesses are reached through their own CiA 720 objects, which chapter 4 fixes:

- **Signed identity read**, C028h. The device signs its identity blob together with the client's challenge under the IDevID, and the client verifies against the device public key.
- **Signed generic read**, C042h:01h. The device signs the target value together with the challenge under the IDevID, and the client verifies against the device public key.
- **Signed generic write**, C042h:02h, and **signed command**, C049h. The client signs the value or the command under its integrator key, and the device verifies against the integrator public key it installed from the ownership voucher.

Freshness comes from the two-pass challenge, in which each side contributes a fresh random that the signed transcript covers, so a captured signature cannot be replayed. The server-issued challenge is also the denial-of-service gate: a node spends a verification only against a challenge it issued itself, so an unsolicited signature costs nothing. A signature mismatch aborts with `FBSEC_ABORT_SIG_VERIFY` (`0xC1`).

The transcript binds the target index and subindex, both peer identifiers, both challenge randoms and the value, under a preamble that separates it by role. Appendix C lists the role bytes and the 32-byte context layout.

11.4 Manufacturer-to-Integrator Handover

Handover follows four steps over the RPK objects listed in chapter 4:

1. **Verify genuineness**, a signed identity read at C028h. The device proves it is genuine with its IDevID.
2. **Claim ownership**, a voucher write at C020h:02h. Authorized model only.
3. **Install the Provisioning Key**, a signed write at C02Fh.
4. **Generate the LDevID**, at C020h:03h, where the request triggers key generation and the reply exports the public key.

> **NOTE:** These operations sit at their CiA 720 RPK addresses. Earlier drafts of SOFA carried them on manufacturer-specific placeholders in the 5FA2h to 5FA6h range; the RPK migration moved them onto the reserved addresses with no legacy aliases left behind, and C000h now advertises the RPK mechanism whenever the layer is built.

What the Device Proves: the Identity Blob

Step 1 answers a 16-byte freshness nonce with 168 bytes in three parts:

Part	Size	Contents
Identity blob	40 bytes	The 8-byte device serial followed by the 32-byte IDevID public key
Manufacturer certificate	64 bytes	An Ed25519 signature by the manufacturer anchor over that blob. It binds the serial to the public key, which is what makes the blob a claim of genuineness rather than an assertion
Device signature	64 bytes	An Ed25519 signature by the device over the blob together with the caller's nonce, proving the device holds the matching private key right now

The certificate is a bare signature over a bare public key, not an X.509 certificate. A constrained CANopen FD device has neither the code space nor the frame budget for a certificate chain, so SOFA carries the same binding in 64 bytes.

Where the Voucher Comes From

The term is borrowed from BRSKI, the IETF (Internet Engineering Task Force) Bootstrapping Remote Secure Key Infrastructure (RFC 8995). BRSKI answers a question every device faces the first time it is powered up in a stranger's network: a device that has met nobody has no way to tell which of the nodes around it is entitled to own it. BRSKI's answer is that the manufacturer says so. The device leaves the factory already knowing its manufacturer's public key, and the manufacturer issues a signed statement, the **voucher**, naming the new owner. The device checks that statement against the key it already holds, and so learns whom to trust without having to trust anything about the network it arrived on. RFC 8366 defines the artifact itself.

SOFA borrows the mechanism and drops the infrastructure. Full BRSKI runs the exchange through an online manufacturer service and a local registrar; SOFA assumes the voucher was issued in advance and is simply presented to the device during commissioning. It also takes the idea and not the encoding: an RFC 8366 voucher is CMS-signed JSON or CBOR, far larger than a fieldbus exchange can carry, so SOFA uses a fixed 108-byte binary equivalent carrying the same four facts.

Field	Size	Purpose
Device serial	8 bytes	Which device this voucher is about. Must match the serial from step 1, so a voucher cannot be replayed onto a different device
Integrator public key	32 bytes	Who the new owner is. This key becomes the anchor the device demands for the Provisioning Key install
Owner epoch	4 bytes	Must strictly increase, so an old voucher cannot return a previous owner
Manufacturer signature	64 bytes	Ed25519 signature over the other three fields, verified against the manufacturer anchor the device holds

The size is simply the sum: 8 plus 32 plus 4 plus 64. Nothing is padded or negotiable.

The Two Models

The models differ in step 2 and in what step 3 demands.

Basic model. Steps 1, 3 and 4 run with trust on first use. The device proves genuineness with its IDevID, then accepts the first Provisioning Key install and generates its LDevID.

Authorized model. Step 2 is added. The device verifies the voucher against its manufacturer anchor and enforces a strictly increasing owner epoch before accepting a new owner. The owner epoch, a read-only counter at C020h:01h, is what prevents rollback to a previous owner. This model also requires the Provisioning Key install to be signed by the owner the voucher named.

The commissioning tool drives the whole flow through the `fbsec_commission_*` functions, and the client exposes it as the `commission <node>` subcommand.

Security of the Provisioning Key Install

The install payload at C02Fh is the 16-byte Provisioning Key followed by a 64-byte signature. Two properties of it need stating plainly.

> **CAUTION: The Provisioning Key travels in the clear.** It is not encrypted, in either model. The signature authenticates the sender; it does not conceal the key. Anyone passively recording the bus during a handover captures the Provisioning Key and, with it, every session key derived from it thereafter.
>
> **In the basic model the signature is not verified at all.** The device accepts the first install it receives, discards the signature unread, and stores the key. Genuineness in step 1 proves the device to the tool, not the tool to the device, so nothing authenticates the installer. Any node that reaches an uncommissioned device first becomes its owner.

The consequence is that handover is only as trustworthy as the environment it runs in. Treat it as a factory or commissioning-bench operation on a physically controlled segment, not as something to run on a live plant network.

The authorized model narrows the second problem but not the first. It refuses an install that is not signed by the owner the manufacturer named, so an arbitrary node can no longer claim the device; the key itself is still plaintext on the wire.

Sending the key in the clear is a design position rather than an omission. EmSA-WP-104 makes the controlled environment the control: the first credential install happens there, and an uncommissioned device accepts nothing else until it does. Wrapping the key to the device's public key would remove the exposure, but it would also add asymmetric key agreement to a path that WP-104 deliberately keeps within reach of devices that cannot run asymmetric cryptography at production rates. Plan handover accordingly, and treat any field re-keying outside a controlled environment as a separate problem this release does not solve.

11.5 Multi-Frame Bodies

A 64-byte signature, the 40-byte identity blob and the 108-byte voucher each exceed one expedited USDO frame, so the asymmetric layer carries them as CiA-1301 USDO segmented transfers. Chapter 5 covers the two properties that most often trip up a port, and the one profile-level liberty SOFA takes. The symmetric verbs remain single-frame.

12
Threat Model

This chapter states what the cryptography defends against, what it does not, and what SOFA claims. Read it before deciding that the demonstrator is fit for a given deployment.

12.1 In Scope

The cryptography defends against the following:

Replay. Every single-shot secure verb carries mutual-random freshness in its associated data, on both reads and writes. Cyclic poll frames inherit the same base through `nonce_base` and add a monotonic counter bound into the nonce. A captured tag does not authorize a second request, because one side's contribution is regenerated on every new arming, so the nonce base never repeats. The two randoms give freshness from both sides, but nonce-collision resistance for the encrypting side rests on that side's own generator: under the threat model one peer is hostile and can hold its contribution constant, so the encrypting side (the server on reads, the client on writes) must guarantee its own random never repeats under a key. A repeated encryptor random reuses a GCM (key, nonce) pair, which is catastrophic. The port random source must be a CSPRNG that does not repeat within a key's lifetime.

Tag forgery. The default configuration uses the full 16-byte GCM tag, so no truncation argument has to be made. The length is configurable down to 4 bytes for deployments that need the frame space; a truncated tag stays inside NIST SP 800-38D's tag-length analysis only for a bounded number of invocations per key, and the per-key frame budget in chapter 10 is what enforces that bound.

Mechanism and key-identifier downgrade. The full mechanism byte and the full wire key identifier are in the associated-data prefix. Flipping the encryption bit or the cyclic-arm bit, repurposing a reserved bit, or rerouting to a different key selector all fail verification.

Direction and addressing tampering. The device id, the `data_id`, the direction code and the payload length are all in the associated data. An attacker cannot redirect a captured tag to a different device, a different object or a different direction.

Role escalation through the crypto layer. Possession of a valid key does not by itself grant an operation. The role hook is consulted after tag verification, so an Operator-role write is refused even though its tag is perfectly valid.

12.2 Out of Scope

The cryptography does not defend against the following. Address them in the platform or the process.

Trust in the simulated bus. The SOFA bus is a broadcast relay. A malicious relay sees every secure frame and can drop or reorder them. Confidentiality is at the cipher when encryption is enabled; availability is not protected. On a real deployment the equivalent question is trust in the physical bus and its gateways.

Hardware side channels. Timing attacks against the AES core, power analysis against the tag computation, and electromagnetic emanation are not addressed. The mbedtls AES path used here is not constant-time on all MCUs. High-assurance deployments should move the AEAD onto a hardware engine, through a secure element or a TrustZone service as described in chapter 10.

Key extraction from a compromised device. If an attacker reads the key array through a debug port, JTAG, fault injection, or a kernel-level exploit on a hosted platform, the tunnel is over. The storage topologies in chapter 10 exist for exactly this; the cryptography itself assumes the keys are confidential.

Supply chain and coercion at provisioning time. Factory injection is only as strong as the factory's process. SOFA does not address supply-chain attestation. The optional asymmetric layer narrows this by proving genuineness against a manufacturer anchor, but it does not eliminate it.

Denial of service. Nothing in the protocol limits how fast a peer may send. A flood of malformed frames costs the server one tag verification each. A narrower point sits alongside this: the armed read and write slots are one per entry, so SOFA assumes single-client (single-manager) access. Concurrent access by a second client is not supported; the specified response is a single-client-access abort rather than silently overwriting the first client's armed state.

Confidentiality in authenticate-only mode. With encryption disabled (`FBSEC_AEAD_ENCRYPTION = 0`) a SECURE_RO payload rides the wire in clear, carried in the associated data. Any node can then drive a secure read and recover the value with no key. A deployment that treats read data as confidential must run encryption mode; authenticate-only provides neither read confidentiality nor read access control.

Device authenticity under a shared key. The associated data binds the peer node ids, but that authenticates a specific device only when the session key is unique to it. If a key is shared across a trust domain, any member can produce a valid tag for any node id, so the binding proves message authenticity, not which device answered, and a compromised member can impersonate any node. Device authenticity in a shared-key deployment requires per-device keys or the RPK layer of chapter 11. Whether keys are shared, per-device or ladder-derived is the integrator's choice, so this is a consequence to weigh, not a mandate.

Session-key derivation binding. The session key binds the key id but takes its salt from the host. Freshness and per-device separation therefore depend on the host supplying a per-device or per-session salt and rotating it; a static, shared salt turns the session key into a long-lived, domain-wide key.

Capability downgrade through the cold descriptor. The C000h and C001h descriptors are read cold, without a tag. An active attacker can spoof them to hide a stronger mechanism, so a client that merely uses whatever is offered can be downgraded onto the plain tunnel. The required security level must be the client's own local policy: if a deployment mandates signed access and the descriptor denies it, the client aborts rather than falling back. The descriptor is an optimization and never lowers the client's configured minimum.

Ownership on first use. In the basic handover model the first party to reach an uncommissioned device can claim it. The residual depends on the claim gate: a well-known token or trust-on-first-use has no cryptographic defense against a first mover; a per-unit printed Device Claim Token requires the first mover to hold that token; a manufacturer-signed voucher closes it against a network attacker. For the token tiers a physically controlled commissioning environment is a normative assumption, not a nicety.

Out-of-band key-management binding. Signed-FBsec verifies each signature against the peer public key obtained from the key-management service, keyed by node identity. The mutual authentication is only as strong as that binding, so it must be established over an authenticated channel; influence the mapping and a substituted key passes the check.

Rollback of ownership. The owner epoch that stops a captured voucher from re-claiming a device must survive a manufacturer reset and resist a direct write. The whole handover replay defense rests on that storage being restore-surviving and tamper-resistant, which is a normative obligation on the device platform.

12.3 What SOFA Claims

The cryptographic primitives are standards-conforming building blocks chosen for portability across embedded crypto libraries:

- HKDF-SHA-256, per RFC 5869 and NIST SP 800-56C Rev. 2.
- AES-128-GCM, per NIST FIPS 197 and SP 800-38D, equivalent to ISO/IEC 19772:2020 mechanism 5.

The entity-authentication pattern aligns with ISO/IEC 9798-2 in a specific and limited sense. Each single-shot access follows the shape of Mechanism 4, three-pass mutual authentication using random challenges, truncated to two passes so that authentication is unilateral by construction. Both peers still contribute a random time-variant parameter, and both feed the nonce. EmSA-WP-105 section 9.1 lists the five named deviations: the dropped third pass, AEAD replacing a bare MAC (Message Authentication Code), explicit nonce construction, entity identifiers carried in associated data, and the counter extension for cyclic continuations.

A secure read authenticates the server to the client, because the server proves possession of the role key by producing a valid tag over the client's random and both peer identifiers. A secure write authenticates the client to the server, in mirror.

A secure read grants the requester no authentication in return. The arm and the empty trigger are unauthenticated, so any node can drive an armed read to completion and receive the reply. Read confidentiality therefore rests entirely on encryption mode and key possession, never on who asked.

12.4 What SOFA Does Not Claim

Three claims are explicitly not made:

- Full conformance to any specific ISO/IEC 9798 standard.
- Mutual entity authentication per an ISO/IEC 9798-2 mechanism. The optional RPK layer of chapter 11 adds Ed25519 signature authentication, but each signed access authenticates one party, the signer, and the signature replaces the AEAD tag rather than adding to it.
- A standalone authenticated-session protocol. Entity authentication is a side effect of authenticated data access; there is no session-establishment handshake independent of a specific data object.

> **CAUTION:** Beyond the protocol, the demonstrator itself makes no security claim. It ships public demonstration keys, holds them in ordinary RAM, and implements none of the WP-104 key lifecycle. Chapter 2 states this and it bears repeating here: the demonstrator is for understanding and porting, not for deployment.

A

Abort and Status Codes

This appendix collects every status value the demonstrator can produce, on both peers and at every layer.

A.1 Abort Codes

An abort is a seven-byte USDO frame as CiA-1301 Table 32 defines it: destination address, command specifier `0x7F`, session, subindex, index little-endian, and a single-byte abort code at offset 6. The code is an `UNSIGNED8`, not the 32-bit value classical CiA-301 SDO uses.

Wherever CiA-1301 Table 31 has an exact match, SOFA sends the standard code.

Code	Table 31 meaning	Raised when
`0x13`	Unknown command specifier	A request command specifier the codec does not implement.
`0x14`	Invalid value in counter byte	The USDO segment counter is not the expected one.
`0x15`	Incorrect data size	A segmented transfer delivered more or fewer bytes than its initiate frame announced.
`0x21`	Session-ID wrong or unknown	A segment or upload request arrived for no open USDO transfer.
`0x31`	Attempt to read a write-only data element	A read of the write-only C01Fh key-set object.
`0x32`	Attempt to write a read-only data element	A body-bearing request to a descriptor, a constant entry, or the C011h key-identifier array.
`0x33`	Data object does not exist	The index is not a registered entry.
`0x34`	Sub-index does not exist	A descriptor subindex above the highest supported, or a `data_id` whose reserved low byte is non-zero.
`0x36`	Data type or length does not match	A length mismatch where the code cannot tell which direction.
`0x37`	Length of service parameter too high	The body is longer than the entry.
`0x38`	Length of service parameter too low	The body is shorter than the entry.
`0x60`	Data cannot be transferred to the application	Internal failure: the random source, an AEAD seal, signing, or a hook returning the wrong length.
`0x62`	Present device state	The access-allowed hook refused, which is to say a lock or write-protect state.

SOFA-Specific Codes

Table 31 has no code for an authentication failure, and CiA-1301 defines no manufacturer-specific range. SOFA therefore assigns its own in `0xC0` to `0xCF`.

Code	Meaning	Raised when
`0xC0`	AEAD tag verification failed	A forged or tampered request, on a single-shot verb or a poll.
`0xC1`	Signature verification failed	An Ed25519 signature did not verify, on a signed (RPK) access, a voucher, or a provisioning install.
`0xC2`	Ownership voucher rejected	The voucher failed against the manufacturer anchor, or its epoch did not increase.
`0xC3`	Role policy denied	The role hook refused this operation for this key tier.
`0xC4`	Key identifier not accepted	A reserved key-identifier bit was set, the key id did not match a bound entry, the slot is unprovisioned, or a cyclic key id arrived on the wrong pass.
`0xC5`	Secure session unknown or expired	No armed slot, a stale challenge, an idle timeout, or the wrong peer.
`0xC6`	Cyclic poll counter desync	SOFA's own poll counter did not match.
`0xC7`	Frame budget reached	The per-key limit was reached and the slot was torn down. Re-arm.
`0xC8`	Feature not compiled in	The read, write or cyclic path was stripped at build time.
`0xC9`	Function not implemented yet	The object exists in the dictionary but its function is deferred: a salt write to C010h, or a key-install write to C01Fh.
`0xCA` to `0xCF`	Reserved for SOFA	Not assigned.

CAUTION: CiA-1301 reserves `0x71` to `0xFF` for its own future use, so the `0xC0` block is borrowed rather than owned. If CiA assigns codes in that range, these values have to move. Treat them as a SOFA profile assignment, not as part of the standard.

NOTE: Two distinctions that the previous scheme collapsed into one code are now visible. `0x14` and `0x21` refer to the **USDO** segment counter and USDO session, which belong to the transport. `0xC6` and `0xC5` refer to SOFA's **cyclic poll** counter and secure session, which belong to the tunnel. A desync in one is a different fault from a desync in the other.

NOTE: A plain, non-secure read or write against a secure entry is rejected below the secure tunnel, by the object-dictionary access layer, and never reaches these codes.

A.2 Dispatch Status

`fbsec_sod_dispatch()` returns one of four values to the variant adapter.

Value	Name	Meaning
0	`FBSEC_SOD_OK`	Handled; a reply payload is ready.
1	`FBSEC_SOD_NOT_HANDLED`	The `data_id` is not a registered secure entry.
2	`FBSEC_SOD_ABORT`	Refused; `out_abort` holds the code.
3	`FBSEC_SOD_DEFER`	Handled; acknowledge with an empty reply and wait for the next pass.

A.3 Client Status

The client secure-protocol layer returns `fbsec_secure_status_t`.

Value	Name	Meaning	Exit
0	`FBSEC_SECP_OK`	Success	0
1	`FBSEC_SECP_TIMEOUT`	No response within the timeout	1
2	`FBSEC_SECP_TX`	Transmit or socket error	1
3	`FBSEC_SECP_PROTOCOL`	Malformed reply envelope, or the key-use limit reached	1
4	`FBSEC_SECP_BUFSIZE`	Response too large for the caller buffer	1
5	`FBSEC_SECP_ABORT`	Server returned an abort	2
6	`FBSEC_SECP_TAG`	AEAD tag verification failed locally	3
7	`FBSEC_SECP_RANDOM`	The random hook returned false	1

A.4 Transport Layer Status

These values never leave the process. They are listed because they appear in verbose traces and in the simulated bus frame-error events.

CAN FD frame validation:

Value	Meaning
0	Valid
1	Reserved CAN identifier bits set
2	Reserved flag bits set
3	Payload length above 64

USDO decode:

Value	Meaning
0	Decoded

Value	Meaning
1	Frame too short
2	Reserved node id
3	Bad command specifier
4	Index below `0x1000`
5	Abort frame with a data length other than 4
6	Not a USDO frame

Carrier status:

Value	Meaning
0	Success
1	Timeout
2	Connection closed
3	Frame error
4	Network error

The simulated bus frame-error codes are listed in chapter 5.

B
Configuration Macro Reference

Compile-time configuration lives in two places. Some knobs are CMake cache variables set with `-D` at configure time; the rest are edited directly in `shared/fbsec_config.h`. This appendix lists both, with defaults and permitted ranges.

B.1 CMake Cache Variables

Set these at configure time, for example `cmake -S . -B build -DFBSEC_FEATURE_ASYM=0` for a minimal AEAD-only device.

Variable	Default	Range	Effect
`FBSEC_FEATURE_READ`	1	0 or 1	Include the read path. With 0, the server rejects read directions and the client lacks the read entry points.
`FBSEC_FEATURE_WRITE`	1	0 or 1	Include the write path. Symmetric with the read knob.
`FBSEC_FEATURE_CYCLIC`	1	0 or 1	Include cyclic sessions. With 0, bit 6 of the wire key selector is treated as reserved and the poll directions are rejected.
`FBSEC_FEATURE_ASYM`	1	0 or 1	Master gate for the Ed25519 identity and RPK layer, on by default. With 0 the wire behavior is byte-identical to a symmetric-only build and none of the asymmetric code is compiled.
`FBSEC_HANDOVER_AUTHORIZED`	1	0 or 1	Select the authorized handover model, adding the ownership voucher and owner epoch. Follows `FBSEC_FEATURE_ASYM` and defaults on with it; set to 0 for the born-open model. Setting it to 1 while `FBSEC_FEATURE_ASYM` is 0 is a fatal error.
`FBSEC_AEAD_DEV_ID_SIZE`	1	1 or 2	Width of each peer identifier in the associated-data prefix. 1 matches CANopen FD node ids; 2 is reserved for future variants. Any other value is a fatal error.
`FBSEC_VARIANT_CANOPEN_FD`	ON	ON or OFF	Build the CANopen FD variant.

B.2 Header Macros

These are edited in `shared/fbsec_config.h` rather than passed to CMake, except where noted as overridable.

Macro	Default	Range	Effect
FBSEC_AEAD_AES128_GCM	1	0 or 1	Select AES-128-GCM. Exactly one AEAD primitive must be 1.
FBSEC_AEAD_AES256_GCM	0	0 or 1	Select AES-256-GCM, which makes the session key 32 bytes.
FBSEC_AEAD_ASCON_128	0	0	Reserved. Enabling it is a build error until Ascon is implemented.
FBSEC_AEAD_CHACHA20_POLY1305	0	0	Reserved. Enabling it is a build error.
FBSEC_KDF_HKDF_SHA256	1	1	The only key-derivation function implemented.
FBSEC_AEAD_TAG_LEN_BYTES	16	4 to 16	AES-GCM tag length. The default is the full tag. Truncating trades forgery margin for frame space and is a deliberate choice, not a default.
FBSEC_AEAD_ENCRYPTION	1	0 or 1	Compile-time default only. 1 is encrypt-and-authenticate, 0 is authenticate-only. The runtime choice is bit 7 of the wire key selector.
FBSEC_AEAD_RANDOM_SIZE	12	12 or more	Bytes of randomness each peer contributes per single-shot transfer. Overridable at configure time. Must be at least the nonce size.
FBSEC_AEAD_KEY_USE_LIMIT	1 000 000	any	AEAD invocations allowed per key and session.
FBSEC_ASYM_ED25519	follows FBSEC_FEATURE_ASYM	0 or 1	Asymmetric backend. Ed25519 is the only one implemented.

B.3 Registry and Buffer Limits

These are defined in `shared/fbsec_secure_od.h` and `shared/fbsec_aead.h`.

Macro	Default	Effect
FBSEC_SOD_MAX_ENTRIES	8	Secure entries the registry holds.
FBSEC_SOD_KEY_SLOTS	4	Server-side key slots.
FBSEC_SOD_CHALLENGE_TIMEOUT_MS	5000	Validity window of a Pass-1 challenge.
FBSEC_SOD_SESSION_IDLE_TIMEOUT_MS	60000	Idle timeout of a cyclic session.
FBSEC_AEAD_MAX_PROTECTED	32	Largest plaintext an entry may declare.
FBSEC_AEAD_KEY_SIZE	16 or 32	Session key size, following the AEAD primitive.
FBSEC_AEAD_NONCE_SIZE	12	GCM nonce size.

Macro	Default	Effect
FBSEC_AEAD_KEYID_MAX	15	Highest base key identifier the wire byte can express.
FBSEC_AEAD_PROTOCOL_VERSION	0x01	Value at offset 0 of the associated-data prefix.

B.4 CiA 720 Security Object Constants

These name the CiA 720 security objects, their limits and the demonstration key-id values. The index and descriptor macros are in `shared/fbsec_descriptor.h` and `server_common/server_common_security.h`; the constant object-dictionary macros are in `server_common/server_common_const_od.h`; the not-implemented abort is in `shared/fbsec_abort.h`; and the demo key-id values are in `server_common/server_common_keys.h`.

Macro	Value	Meaning
FBSEC_DESC_CAP_INDEX	0xC000	Security profile and capabilities record.
FBSEC_DESC_STAT_INDEX	0xC001	Security status record.
FBSEC_DESC_CAP_SUB_MAX	0x07	Highest C000h subindex.
FBSEC_DESC_STAT_SUB_MAX	0x02	Highest C001h subindex.
FBSEC_SEC_INDEX_SESSION_SALT	0xC010	Session pre-requisites, surface-only this pass.
FBSEC_SEC_INDEX_KEY_IDS	0xC011	AEAD key identifiers array.
FBSEC_SEC_INDEX_KEY_SET	0xC01F	Key set, surface-only this pass.
FBSEC_OD_IDENTITY_INDEX	0x1018	CANopen identity object, served from the object-dictionary file.
FBSEC_CONST_OD_MAX_ENTRIES	32	Constant object-dictionary table capacity.
FBSEC_CONST_OD_MAX_DATA	8	Maximum bytes per constant entry.
FBSEC_ABORT_NOT_IMPLEMENTED	0xC9	Object present, function not implemented yet.
FBSEC_DEMO_KEYID_VALUE_PROVISIONING	0x1001	C011h id for the demo Provisioning key.
FBSEC_DEMO_KEYID_VALUE_INTEGRATOR	0x1002	C011h id for the demo Integrator key.
FBSEC_DEMO_KEYID_VALUE_OPERATOR	0x1003	C011h id for the demo Operator key.

B.5 Build-Time Assertions

The header refuses to compile an inconsistent configuration rather than producing a binary that fails at run time. The checks are:

- Exactly one `FBSEC_AEAD_*` primitive is 1.
- Exactly one `FBSEC_KDF_*` is 1.
- For AES-GCM, the tag length is between 4 and 16 bytes.
- At least one of `FBSEC_FEATURE_READ` and `FBSEC_FEATURE_WRITE` is 1.
- `FBSEC_AEAD_RANDOM_SIZE` is at least the nonce size.
- `FBSEC_AEAD_DEV_ID_SIZE` is 1 or 2.

- The asymmetric knobs are internally consistent, and `FBSEC_HANDOVER_AUTHORIZED` is not set without `FBSEC_FEATURE_ASYM`.

> **NOTE:** Peers built with different configurations do not negotiate. The mechanism byte and the peer-identifier width are both part of the associated data, so a mismatch surfaces as a clean tag-verify failure on the first secure verb. That is the intended failure mode, and it is why the protocol version does not change when these knobs do.

C
Demo Keys and Identities

> **CAUTION:** Every value in this appendix is public by virtue of being printed here and of shipping in the source tree. These are demonstration credentials, not a security claim. A real deployment installs its own keys through the host's key store and never reuses anything below.

C.1 Demonstration Session Keys

The reference client and server both carry the same three hard-coded session keys, and `run/keys-demo.txt` holds the same values in key-file form. They are declared as 32-byte literals so that one source compiles for both AES-128-GCM and AES-256-GCM; on an AES-128-GCM build only the first sixteen bytes are used.

Key id	Role	C011h id	First 16 bytes
1	Provisioning Key	`0x1001`	`00 11 22 33 44 55 66 77 88 99 AA BB CC DD EE FF`
2	Integrator Key	`0x1002`	`AA BB CC DD EE FF 00 11 22 33 44 55 66 77 88 99`
3	Operator Key	`0x1003`	`FF EE DD CC BB AA 99 88 77 66 55 44 33 22 11 00`

The C011h id is the non-secret key id or version the server reports for each slot, set from the fourth column of the key file. The demonstration uses `0x1001` through `0x1003` deliberately, so the reported id is visibly independent of the slot and role number. On an AES-256-GCM build each key is its sixteen bytes repeated once, giving 32 bytes.

The shipped `run/keys-demo.txt` is:

```
# keyid  role-label      key (hex)                          id
1        prov-session    00112233445566778899AABBCCDDEEFF   0x1001
2        integ-session   AABBCCDDEEFF00112233445566778899   0x1002
3        op-session      FFEEDDCCBBAA99887766554433221100   0x1003
```

> **CAUTION:** The server loads `--key-file` first and treats every slot as write-once, then backfills the demonstration keys into any slot the file left empty. A partial key file therefore leaves the public demonstration keys live in the remaining slots. Provision every slot the deployment uses, or remove the backfill for a build that must not carry them.

C.2 Demonstration Identity (Object 1018h)

The constant object-dictionary file `run/sofa-od-demo.txt` defines object 1018h, the CANopen identity that C018h returns under an AEAD tag. The server loads it with `--od-file`, and `start_fd_server.bat` passes it. The four values are:

Sub	Field	Value
0x01	Vendor id	`0x000002F1`
0x02	Product code	`0x534F4641`, the ASCII string "SOFA"
0x03	Revision	`0x00010001`
0x04	Serial number	`0x00000001`

CAUTION: The vendor id `0x000002F1` is a demonstration placeholder, not a CiA-assigned vendor id. A real device must carry its own assigned vendor id. Without an `--od-file` supplying object 1018h the identity is absent and C018h aborts.

C.3 Demonstration Ed25519 Material

The asymmetric layer, when built, uses fixed demonstration seeds so that a handover can be walked through repeatably. Each seed is a 32-byte ascending run.

Identity	Seed
Manufacturer trust anchor	`0x10` through `0x2F`
IDevID, the factory device identity	`0x30` through `0x4F`
Integrator owner key	`0x50` through `0x6F`
LDevID, the local device identity	`0x70` through `0x8F`

Two further fixed values complete the demonstration identity:

- The device serial is `53 4F 46 41 00 00 00 01`, which is the ASCII string `SOFA` followed by `00 00 00 01`.
- The owner epoch starts at 1.

Ed25519 key pairs are derived deterministically from the seed, so publishing the seed publishes the private key. This material exists only so that the demonstration is repeatable.

C.4 Sizes of the Asymmetric Material

For reference when sizing buffers on a target:

Item	Size
Public key	32 bytes
Private key	64 bytes, the seed followed by the public key
Seed	32 bytes
Signature	64 bytes
Device serial	8 bytes
Identity blob	40 bytes, the serial followed by the IDevID public key

Item	Size
Identity-read nonce	16 bytes
Ownership voucher	108 bytes
RPK signed-access context	32 bytes: `data_id[4]`, server[2], client[2], `client_random[12]`, `server_random[12]`. The signed value is appended to this context.

Signature transcripts are domain-separated by a two-byte preamble of the protocol version `0x01` followed by a role byte, so a signature produced for one purpose cannot be replayed as another.

Role byte	Purpose
`0x01`	Identity read, the genuineness proof
`0x02`	Provisioning-key install acknowledgement
`0x03`	LDevID export
`0x04`	Signed generic write or command, client to device
`0x05`	Signed generic read, device to client
`0x06`	Ownership voucher, signed by the manufacturer anchor
`0x07`	IDevID certificate

D
Reference Files

SOFA is documented across three documents plus the source tree. This appendix says which one owns what, so a reader knows where an unanswered question belongs.

D.1 Document Map

Document	Owns
EmSA-WP-105, Secure Object Fieldbus Access	The conceptual model and the protocol shape: what a secure read and a secure write are, why authentication is unilateral, the tag and nonce construction in the abstract, the standards alignment, and the optional asymmetric identity model. Normative for shape, silent on concrete sizes.
EmSA-WP-104, Key Provisioning for Minimal Fieldbus Systems	The key lifecycle: roots of trust, the uncommissioned state and device claim token, device-unique provisioning keys, ownership claim at handover, integrator keys, manufacturer reset, salt lifecycle and session-key derivation. This is what the demonstrator deliberately skips.
This manual, EmSA-UM-105-COP	Everything the CANopen FD demonstrator fixes: the implemented object dictionary, the exact wire bytes, the build configuration, how to run it, the command line, and the porting guide. Chapter 4 is the normative object-dictionary reference and chapter 5 is the normative wire reference.

Chapters 4 and 5 own the concrete implementation: chapter 4 fixes every implemented index and subindex, and chapter 5 fixes every byte on the wire, the associated data, the key identifier and the flows. Where this manual and the source tree disagree, the code is the arbiter, and the discrepancy is a defect in the manual. Where this manual and a whitepaper disagree, the whitepaper describes the general model and this manual describes one implementation of it.

D.2 Source Tree

The layers are separated so a future variant can be added without touching the common code.

Path	Contents
`shared/`	Variant-neutral core: AEAD shim, HKDF, the secure-object registry and dispatch, the client secure-protocol state machine, the descriptor codec, and the asymmetric layer.
`server_common/`	Server-side glue: command line, key store, object-dictionary setup, port hooks, dispatch, trace, handover.
`client_common/`	Client-side glue: command line, key store, verbs, interactive menu, trace, platform hooks, commissioning.
`bus_common/`	The TCP relay used by every variant's bus simulator.

Path	Contents
`variants/canopen_fd/`	The CANopen FD variant: address mapping, USDO codec, frame codec, carrier, bus simulator with the PEAK bridge, server and client executables.
`third_party/`	Vendored mbedtls subset and, for asymmetric builds, Monocypher.
`run/`	Launcher batches and the demonstration key and object-dictionary files.

D.3 Standards Referenced

Standard	Used for
NIST FIPS 197	AES block cipher
NIST SP 800-38D	GCM mode, tag truncation and the invocation bound
ISO/IEC 19772:2020	AES-GCM as authenticated-encryption mechanism 5
RFC 5869 and NIST SP 800-56C Rev. 2	HKDF-SHA-256
RFC 8032	Ed25519 signatures
ISO/IEC 9798-2	The entity-authentication pattern the two-pass exchange follows
CiA-1301	CANopen FD application layer: object dictionary layout, the manufacturer-specific index range, the USDO protocol including segmented transfer, and the abort frame and codes
CiA-301	Classical SDO framing, for the fallback mapping only
IEEE 802.1AR	The IDevID and LDevID identity model
RFC 8995	BRSKI, the ownership-voucher model the handover borrows from
RFC 8366	The voucher artifact BRSKI uses, which SOFA replaces with a compact binary equivalent